

GIT & GITHUB

The Practical Guide Book

From your first commit to team workflows.
Theory and hands-on practice, step by step,
for Git Bash on Windows and Terminal on macOS.

EmpowerWithIT

Educate, Innovate, Empower.

How to use this book

This book teaches Git and GitHub from zero. It is written for students who will **type along**. Every idea is explained in plain language first (the theory), and then you practise it immediately in a hands-on lab, with the exact commands and the output you should expect to see.

The rule: read a little, type a lot

Reading about Git is like reading about swimming. You only learn by getting in the water. Open a terminal beside this book, and run every lab.

What the boxes mean

Box	Meaning
KEY IDEA	The one thing from this section you must remember.
ANALOGY	A real-life comparison to make a concept click.
WINDOWS / macOS	Differences between Git Bash on Windows and Terminal on macOS. Everything else works the same on both.
PRO TIP	Habits that professionals use.
WARNING	Commands or situations that can lose work. Read these twice.
HANDS-ON LAB	A guided exercise. Commands in dark blocks; lines starting with a green $ are what you type. Lines without a $ are the output you should see.
CHECK YOURSELF	Questions at the end of each chapter. Answers are in Appendix E.

About the dark terminal blocks

```
TERMINAL (Git Bash or macOS Terminal)

$ git status                    # a command you type (do not type the $)
On branch main                  # output you will see
nothing to commit, working tree clean
```

Your output will often differ in small ways: your username, the folder path, and especially the **commit hashes** (the short codes like 3f2a9c1), which are unique to your machine. That is normal. What matters is that the *shape* of the output matches.

Suggested path

If you are...	Do this
Brand new to Git	Read Chapters 1 to 8 in order, doing every lab. Then Chapters 9 and 10 (GitHub). Aim for one or two chapters a day.
Comfortable with commits, new to GitHub	Skim Chapters 1 to 6, then work carefully through 7 to 10.
Working in a team already	Jump to Chapters 10 to 13 (collaboration, workflows, advanced tools, Actions), and use Chapter 17 as a troubleshooting reference.
Preparing for a job	Finish the book, then do the three capstone projects in Appendix D and put them on your GitHub profile.

NOTE Windows users: use **Git Bash** (installed with Git for Windows), not Command Prompt or PowerShell, so the commands in this book work exactly as shown. macOS users: use the built-in **Terminal** app.

Contents

How to use this book 2

Chapter 1 Why Git and GitHub? 4

Chapter 2 Setting up your workstation 7

Chapter 3 How Git thinks 11

Chapter 4 Your first repository 14

Chapter 5 Undoing mistakes safely 19

Chapter 6 Branching 23

Chapter 7 Merging and resolving conflicts 26

Chapter 8 Rebasing and tidying history 30

Chapter 9 GitHub and remote repositories 33

Chapter 10 Collaborating: issues, forks, and pull requests 38

Chapter 11 Team workflows and conventions 42

Chapter 12 Advanced tools 45

Chapter 13 Automation with GitHub Actions 49

Chapter 14 GitHub beyond code 52

Chapter 15 Security and professional hygiene 55

Chapter 16 Using AI with Git and GitHub, responsibly 57

Chapter 17 Troubleshooting guide 59

Chapter 18 Habits of professionals 61

Appendix A: Command quick reference 62

Appendix B: Terminal cheat sheet: Git Bash vs macOS 64

Appendix C: Glossary 65

Appendix D: Capstone projects 66

Appendix E: Answer key 67

Appendix F: Resources for next steps 72

CHAPTER 1

Why Git and GitHub?

In this chapter you will
- understand the problem that version control solves
- learn where Git and GitHub came from
- tell the difference between Git and GitHub
- see why every developer is expected to know them

1.1 The problem: files change, people collide

Every project goes through changes. Without a system, this is what happens:

```
project/
  report.docx
  report_v2.docx
  report_final.docx
  report_final_REAL.docx
  report_final_REAL_use_this_one.docx
```

Now add four teammates. Someone emails a zip file. Someone overwrites someone else's changes. On Monday nobody can say which version is current, who changed a line, why it was changed, or how to get back to the version that worked last Tuesday.

A **version control system (VCS)** solves all of this. It records every change to your files over time, so you can:

- **go back** to any earlier version, like a time machine;
- **compare** two versions and see exactly what changed;
- **see who changed what, when, and why**;
- **work in parallel** with other people and combine the work safely;
- **experiment fearlessly**, because anything can be undone.

1.2 Three generations of version control

Generation	Examples	How it works	Weakness
Local	SCCS, RCS	A database of changes on one computer	No collaboration
Centralised	CVS, Subversion (SVN), Perforce	One central server holds the history; developers check files out and in	Needs the network; the server is a single point of failure
Distributed	**Git**, Mercurial	Every developer has a complete copy of the entire history	A little more to learn at first

1.3 A short history

The Linux kernel story. The Linux kernel is one of the largest collaborative projects in the world. For years its developers used a proprietary tool called BitKeeper. In April 2005 the free licence was withdrawn after a dispute, and the community suddenly needed a new tool.

Linus Torvalds, who created Linux, wrote Git himself, and within days it could manage its own code. His goals were: be *fast*, keep the design *simple*, support thousands of *parallel branches*, be *fully distributed*, and cope with huge projects. In the summer of 2005 he handed day-to-day maintenance to **Junio Hamano**, who still leads the project.

NOTE 'Git' is British slang for an unpleasant person. Torvalds joked that he names his projects after himself: first Linux, then Git.

Year	Milestone
2002	The Linux kernel adopts BitKeeper
April 2005	BitKeeper's free licence ends; Linus Torvalds starts Git
December 2005	Git 1.0 released
2008	GitHub launches, founded by Tom Preston-Werner, Chris Wanstrath and PJ Hyett
2018	Microsoft announces its acquisition of GitHub; GitHub Actions is announced
2019	GitHub Actions becomes generally available; Git 2.23 adds **git switch** and **git restore**
2020	The default branch name moves from **master** to **main** on GitHub and in many projects
2021	GitHub stops accepting account passwords for Git operations (tokens or SSH keys are required)
Today	Git is the overwhelming standard; GitHub is home to well over 100 million developers

1.4 Git versus GitHub

The most common confusion for beginners:

KEY IDEA **Git** is a tool on your computer that tracks changes. **GitHub** is a website that hosts Git repositories and adds collaboration features on top.

ANALOGY Email is a technology; Gmail is a service built on it. Git is the technology; GitHub is one service (like GitLab and Bitbucket) built around it. Or: Git is the engine, GitHub is the garage where everyone parks and shares their cars.

	Git	GitHub
What it is	Version control software	A cloud platform that hosts Git repositories
Where it runs	On your computer (terminal or editor)	In the browser, plus the gh command line tool
Needs internet?	No (only to share changes)	Yes
Main job	Record history, branch, merge	Share, review, track issues, automate tests, host websites
Use one without the other?	Yes, Git works fully offline	No, it exists to host Git repositories
Alternatives	Mercurial, Subversion (older)	GitLab, Bitbucket, Azure DevOps, Gitea, Codeberg

1.5 The essence: why every developer uses them

KEY IDEA Git gives your code a **time machine** and **parallel universes**. GitHub gives it a **shared home**, a **review process**, and a **robot assistant**.

Reason	In practice
A safety net	Every commit is a save point. A mistake costs minutes, not days.
Fearless experiments	Branches are cheap. Try risky ideas on a branch; delete it if it fails.
Collaboration	Many people change one project at once and combine the work safely.
Accountability	Every line has an author, a date, and a message explaining it.
Code review	Pull requests mean a second pair of eyes before code reaches the main branch.
Automation	GitHub Actions runs tests and deployments on every push.

Reason	In practice
Backup	Every clone is a full copy. Lose a laptop, clone again.
Career	Your public repositories are your portfolio. Nearly every software job expects Git.

Chapter summary

- Version control records every change so you can go back, compare, and collaborate.
- Git (2005, Linus Torvalds) is the tool; GitHub (2008) is a platform built around it.
- Git is distributed: every copy holds the full history.

CHECK YOURSELF

1. In one sentence, what is the difference between Git and GitHub?
2. What does it mean that Git is 'distributed'?
3. Why was Git created?
4. Name three benefits of using version control.

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 2

Setting up your workstation

In this chapter you will
- open a terminal and understand what the prompt means
- install Git on Windows (Git Bash) or macOS
- learn the 12 terminal commands you need every day
- configure Git with your identity and an editor

You will need: a computer, internet, and about 30 minutes

2.1 What you will use

Tool	Purpose	Where to get it
Terminal	Where you type Git commands	Git Bash (Windows, installed with Git) or Terminal (macOS, built in)
Git	The version control tool itself	git-scm.com, or your package manager
VS Code	A friendly code editor with Git built in	code.visualstudio.com
GitHub account	Hosting and collaboration (used from Chapter 9)	github.com (free)

2.2 Opening a terminal

A **terminal** lets you control the computer by typing commands instead of clicking. The program inside it that reads your commands is called the **shell**. Git was designed for the command line, so learning it there teaches you what is really happening. Editors and apps just press the same buttons for you.

WINDOWS (Git Bash)	macOS (Terminal)
Install Git for Windows first (next section). Then open **Git Bash** from the Start menu, or right-click inside any folder and choose **Open Git Bash here**.	Press **Cmd + Space**, type **Terminal**, and press Enter. (Optional: install iTerm2 or use the terminal inside VS Code.)

The **prompt** is the text before your cursor. It waits for you to type. In this book we show it as a simple **$**.

```
WHAT YOUR PROMPT MAY LOOK LIKE

student@LAPTOP MINGW64 ~          # Windows (Git Bash) prompt looks like this
$

student@MacBook-Air ~ %           # macOS (zsh) prompt looks like this
```

2.3 Installing Git

WINDOWS (Git Bash)	macOS (Terminal)
1. Go to **git-scm.com** and download **Git for Windows**. 2. Run the installer. Most defaults are fine. Choose: **Visual Studio Code** as the default editor; **Override the default branch name** and type **main**; **Git from the command line and also from 3rd-party software** for PATH. 3. Finish, then open **Git Bash**. Shortcut: `winget install --id Git.Git -e --source winget`	1. Open Terminal and type `git --version`. 2. If Git is missing, macOS offers to install the **Xcode Command Line Tools**. Click Install and wait. (Or run `xcode-select --install`.) 3. Alternative with Homebrew: `brew install git`.

```
TERMINAL (Git Bash or macOS Terminal)

$ git --version
git version 2.45.1
```

> **NOTE** Your version number will be different. Anything from 2.30 onward supports every command in this book.

2.4 Terminal survival kit

You only need a handful of commands. They behave the same in Git Bash and macOS Terminal.

Command	What it does
`pwd`	**P**rint **w**orking **d**irectory: shows the folder you are in right now.
`ls`	**L**i**s**t files and folders here. **ls -a** includes hidden ones; **ls -l** shows details.
`cd folder`	**C**hange **d**irectory into *folder*. **cd ..** goes up one level; **cd ~** goes to your home; **cd -** goes back.
`mkdir name`	**M**a**k**e a new **dir**ectory (folder).
`touch file`	Create an empty file (or update its timestamp).
`echo "text" > file`	Write text into a file, replacing what was there. Use **>>** to append instead.
`cat file`	Print a file's contents on screen.
`cp a b / mv a b`	Copy a to b / move or rename a to b.
`rm file`	Delete a file. **rm -r folder** deletes a folder and everything in it. No recycle bin!
`clear`	Clear the screen (shortcut: Ctrl + L).
`code .`	Open the current folder in VS Code.
`history`	Show commands you typed before.

> **PRO TIP** **Tab** completes names for you (type `cd gi` then Tab). The **Up arrow** recalls your previous command. **Ctrl + C** stops a running command. These three habits save hours.

> **WARNING** `rm` and `rm -r` permanently delete. Never run `rm -rf` on a path you have not double-checked.

WINDOWS (Git Bash)	macOS (Terminal)
Paths look like `/c/Users/you/git-book` (the C: drive is **/c**). Always use forward slashes. Copy and paste with **Ctrl + Insert / Shift + Insert** or right-click. Open the current folder in Explorer with `explorer .`	Paths look like `/Users/you/git-book`. Copy and paste with **Cmd + C / Cmd + V**. Open the current folder in Finder with `open .`

HANDS-ON LAB 2.1 Navigate and create files

Create the folder where you will do **every lab in this book**.

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~
$ pwd
/c/Users/student                  # on macOS: /Users/student
$ mkdir git-book
$ cd git-book
$ mkdir practice
$ cd practice
$ echo "Hello terminal" > hello.txt
$ ls
hello.txt
$ cat hello.txt
Hello terminal
$ echo "A second line" >> hello.txt
$ cat hello.txt
Hello terminal
A second line
$ cd ..
$ ls
practice
$ rm -r practice
$ ls
```

If every command worked and the final **`ls`** printed nothing, you are ready. You created, edited, listed, and deleted files from the keyboard.

2.5 Telling Git who you are

Every commit records its author, so Git needs your name and email once per computer. Use the **same email as your GitHub account** (we will create it in Chapter 9) so that your commits are linked to your profile.

HANDS-ON LAB 2.2 Configure Git

```
TERMINAL (Git Bash or macOS Terminal)

$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
$ git config --global core.editor "code --wait"
$ git config --global --list
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
core.editor=code --wait
```

WINDOWS (Git Bash)	macOS (Terminal)
Also run: **`git config --global core.autocrlf true`** (Windows uses CRLF line endings; this keeps your repositories consistent with Mac and Linux teammates.) Git for Windows already includes the Git Credential Manager for GitHub sign-in.	Also run: **`git config --global core.autocrlf input`** and **`git config --global credential.helper osxkeychain`** (stores GitHub credentials in the macOS Keychain).

The **`--global`** flag means 'for every repository on this computer'. Settings live in a file called **.gitconfig** in your home folder.

Choosing an editor (and escaping Vim)

Sometimes Git opens a text editor, for example when you commit without `-m`. If you did not set VS Code as above, Git may open **Vim**, which confuses everyone the first time. If you are stuck in it:

```
IF YOU GET STUCK IN AN EDITOR

# Vim: press Esc, then type  :wq  and Enter   (save and quit)
# Vim: press Esc, then type  :q!  and Enter   (quit without saving)
# nano: Ctrl + O then Enter to save, Ctrl + X to exit
```

To switch to VS Code now: **`git config --global core.editor "code --wait"`**. On macOS, if **`code`** is not found, open VS Code, press Cmd+Shift+P, and run *Shell Command: Install 'code' command in PATH*.

Chapter summary

- A terminal runs commands; the prompt waits for you. Use Git Bash on Windows and Terminal on macOS.
- Learn pwd, ls, cd, mkdir, touch, echo, cat, cp, mv, rm. Tab completes; Up recalls; Ctrl+C stops.
- Run git config once to set your name, email, default branch, and editor.

CHECK YOURSELF

1. Which command shows the folder you are currently in?
2. What is the difference between > and >> when writing to a file?
3. Why does Git need your name and email?
4. You are stuck in Vim. How do you quit and save?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 3

How Git thinks

In this chapter you will
• understand snapshots, commits, and hashes
• see why branches and HEAD are just pointers
• learn the four places your changes travel through
• peek inside the hidden .git folder yourself

You will need: a terminal and Git installed

You can use Git by memorising commands. You will use it **well** if you understand the model underneath. This chapter is short, and it makes every later chapter easier.

3.1 Snapshots, not differences

Some older tools stored a list of changes per file. Git instead takes a **snapshot** of your whole project every time you commit. To stay efficient, files that did not change are simply linked to their previous stored copy.

ANALOGY — A commit is a **save point in a video game**. You can always return to any save point, and you can start a new play-through (a branch) from any of them.

3.2 Commits and hashes

Each commit has: a **snapshot** of the files, an **author**, a **date**, a **message**, and a link to its **parent** commit. Each commit is identified by a **hash**, a 40-character fingerprint such as `3f2a9c1e7b...`. In practice, the first 7 characters are enough.

The hash is *calculated from the content*. Change one character anywhere in history and every hash after it changes. That is how Git detects corruption and tampering.

```
A <-- B <-- C <-- D           each commit points back to its parent
```

3.3 Branches and HEAD are just pointers

```
A---B---C---D         <- main
         \
          E---F       <- feature/login
```

- A **branch** is a tiny file holding the hash of one commit. Creating a branch copies nothing, so branches are cheap and you should use them freely.
- **HEAD** means 'where I am now'. Usually it points at the current branch; when you commit, that branch label moves forward.
- **Detached HEAD** means HEAD points straight at a commit instead of a branch. It is safe to look around there, but make a branch before committing or your work can be lost.

3.4 The four places

Almost every Git command moves changes from one place to another. Memorise this picture, because it is the map for the whole book.

```
WORKING DIR --git add--> STAGING --git commit--> LOCAL REPO --git push--> REMOTE (GitHub)
(your files)             (next commit)           (.git history)            (shared copy)

     <----------------------- git pull / git fetch ----------------------------------
```

Place	Think of it as	Notes
Working directory	Your desk, where you edit files	What you see in your folder and editor
Staging area (index)	A box where you pack the next parcel	You choose exactly which changes go into the next commit
Local repository	Your private, complete history	Stored in the hidden .git folder
Remote repository	The shared copy everyone agrees on	Usually on GitHub; called **origin** by default

ANALOGY Staging is like **packing a box before you post it**. You decide what goes in the box (**add**), you seal and label it (**commit**), and you post it to the shared warehouse (**push**).

3.5 Distributed: every clone is complete

When you clone a repository you get all of its history, not just the latest files. You can commit, branch, and inspect history on a plane with no internet. GitHub is simply the agreed place where people share their copies.

HANDS-ON LAB 3.1 Look inside the .git folder

Let us open the hood. Do not worry about every detail; the point is to see that Git is just files.

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book
$ mkdir inside-git && cd inside-git
$ git init
Initialized empty Git repository in /c/Users/student/git-book/inside-git/.git/
$ ls -a
.  ..  .git
$ ls .git
HEAD  config  description  hooks  info  objects  refs
$ cat .git/HEAD
ref: refs/heads/main
```

The **`.git`** folder *is* the repository. Delete it and your project is ordinary files again. Now make a commit and examine it:

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "Hello" > hello.txt
$ git add hello.txt
$ git commit -m "first commit"
[main (root-commit) 1a2b3c4] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 9f4d96d...
author Your Name <you@example.com> 1760000000 +0300
committer Your Name <you@example.com> 1760000000 +0300

first commit
```

A commit points to a **tree** (the snapshot of your folders), and a tree points to **blobs** (the file contents). Here is something magical: the hash depends only on content, so *everyone in the world gets the same hash* for the same text.

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "test content" | git hash-object --stdin
d670460b4b4aece5915caf5c68d12f560a9fe3e4
```

NOTE Your output will match exactly, and so will your classmates'. Git's four object types are **blob** (file contents), **tree** (a folder), **commit** (a snapshot plus metadata), and **tag** (a permanent label).

Chapter summary

- A commit is a snapshot with an author, message, and parent, identified by a content-based hash.
- Branches and HEAD are pointers, so branching is cheap.
- Changes travel: working directory, then staging, then local repository, then remote.
- The .git folder is the repository. Every clone has the whole history.

CHECK YOURSELF

1. What are the four places a change can live, in order?
2. Why are branches cheap in Git?
3. What does HEAD mean?
4. What happens to the hashes of later commits if you change an old commit?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 4

Your first repository

In this chapter you will
- create a repository and make your first commits
- read the output of git status, log, and diff
- write commit messages that help your future self
- keep unwanted files out of Git with .gitignore

You will need: Chapters 2 and 3 done; a terminal open in ~/git-book

4.1 The everyday loop

You will repeat one loop hundreds of times in your career. Learn it until it is automatic.

```
edit files  -->  git status  -->  git add  -->  git commit  -->  (repeat)
```

KEY IDEA **status, then act, then status again.** `git status` is your map. It costs nothing and tells you what Git thinks is going on. Run it constantly.

Command	What it does
`git init`	Turns the current folder into a Git repository (creates the hidden .git folder).
`git status`	Shows changed, staged, and untracked files and which branch you are on.
`git add <file>`	Stages a file: puts its current content into the next commit.
`git add .`	Stages everything changed in this folder and below.
`git commit -m "message"`	Saves everything staged as a commit with your message.
`git log --oneline`	Shows history, one line per commit.
`git diff`	Shows changes you have **not** staged yet.
`git diff --staged`	Shows what is staged and will go into the next commit.

HANDS-ON LAB 4.1 Create a repository and make a commit

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book
$ mkdir hello-git && cd hello-git
$ git init
Initialized empty Git repository in /c/Users/student/git-book/hello-git/.git/
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Create a file. Git notices it but does not track it yet (it is **untracked**):

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "Hello Git" > hello.txt
$ git status
On branch main
```

```
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Stage it (put it in the box), then commit it (seal the box):

```
TERMINAL (Git Bash or macOS Terminal)
$ git add hello.txt
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt

$ git commit -m "feat: add hello file"
[main (root-commit) 3f2a9c1] feat: add hello file
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
$ git status
On branch main
nothing to commit, working tree clean
```

working tree clean means your files match your last commit. Now look at the history:

```
TERMINAL (Git Bash or macOS Terminal)
$ git log
commit 3f2a9c1e7b4d5a6c8e9f0a1b2c3d4e5f6a7b8c9d (HEAD -> main)
Author: Your Name <you@example.com>
Date:   Mon Oct 12 08:30:00 2026 +0300

    feat: add hello file
$ git log --oneline
3f2a9c1 (HEAD -> main) feat: add hello file
```

> **NOTE** The hash **`3f2a9c1`** will be different on your computer. That is expected. The commit's author, date, and message are all stored with it.

4.2 Why staging exists

A day of work often contains two unrelated changes, such as a bug fix and a new feature. Staging lets you commit them **separately**, so each commit tells one clear story and can be reviewed or undone on its own.

HANDS-ON LAB 4.2 See changes, stage selectively, commit twice

```
TERMINAL (Git Bash or macOS Terminal)
$ echo "Second line" >> hello.txt
$ echo "Welcome to the project" > readme.txt
$ git status -s
 M hello.txt
?? readme.txt
```

`git status -s` is the short form: **M** = modified, **??** = untracked, and the column position tells you staged (left) versus unstaged (right). Inspect the change before committing it:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git diff
diff --git a/hello.txt b/hello.txt
index 8d0e412..9a3b1c5 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
 Hello Git
+Second line
```

Lines starting with **+** were added and with **-** were removed. Now make **two separate commits**:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git add hello.txt
$ git diff
$ git diff --staged
diff --git a/hello.txt b/hello.txt
...
+Second line
$ git commit -m "docs: add second line to hello"
[main 7c1d4e2] docs: add second line to hello
 1 file changed, 1 insertion(+)
$ git add readme.txt
$ git commit -m "docs: add readme"
[main 5b4a3c2] docs: add readme
 1 file changed, 1 insertion(+)
 create mode 100644 readme.txt
$ git log --oneline
5b4a3c2 (HEAD -> main) docs: add readme
7c1d4e2 docs: add second line to hello
3f2a9c1 feat: add hello file
```

Notice that after staging, plain **`git diff`** printed nothing (nothing is left unstaged) while **`git diff --staged`** showed the staged change.

> **PRO TIP** **`git add -p`** lets you stage a file **chunk by chunk**. Git shows each chunk and asks *Stage this hunk? [y,n,q,a,d,s,e,?]*. Answer **y** (yes), **n** (no), or **s** (split the chunk smaller). It is the secret to clean, focused commits.

4.3 Writing good commit messages

A commit message is a note to your future self and your teammates. Compare:

Weak	Strong
fixed stuff	fix: handle empty cart on checkout
changes	feat: add password reset email
update	docs: explain how to run the tests
asdf	refactor: split auth service into two modules

- **Format: `type: short summary`**. Common types are **feat, fix, docs, style, refactor, test, chore** (this is called Conventional Commits; see Chapter 11).
- **Imperative mood:** 'add', not 'added' or 'adds'. Read it as: *If applied, this commit will... add a password reset.*
- **Keep the summary around 50 characters.** For more detail add a blank line and a body: **`git commit -m "summary" -m "longer explanation of why"`**
- **Explain why, not only what.** The diff already shows what changed.

4.4 Telling Git what to ignore

Some files should never be in a repository: passwords, dependency folders like **`node_modules/`**, log files, build output, and system junk such as **`.DS_Store`**. List patterns for them in a file named **.gitignore**.

HANDS-ON LAB 4.3 Create a .gitignore

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "password=12345" > .env
$ echo "debug output" > debug.log
$ git status -s
?? .env
?? debug.log
$ printf ".env\n*.log\n" > .gitignore
$ git status -s
?? .gitignore
$ git add .gitignore
$ git commit -m "chore: add gitignore"
[main 2d1c0b9] chore: add gitignore
 1 file changed, 2 insertions(+)
 create mode 100644 .gitignore
```

Git now ignores the secret and the log, and the ignore rules themselves are version-controlled and shared with your team.

Pattern	Meaning
`*.log`	any file ending in .log
`node_modules/`	a whole folder
`.env`	one specific file
`!important.log`	an exception: do NOT ignore this file
`build/*.js`	all .js files directly inside build/

WARNING .gitignore only affects files that are **not yet tracked**. If you already committed a file, ignoring it later does nothing. Stop tracking it with **`git rm --cached <file>`** (it stays on disk) and commit that. Also remember: a password committed once is in history forever; see Chapter 15.

PRO TIP github.com/github/gitignore has ready-made templates for almost every language and framework.

4.5 Renaming and deleting files

Command	What it does
`git mv old new`	Renames or moves a file and stages the change.
`git rm file`	Deletes a file and stages the deletion.
`git rm --cached file`	Stops tracking a file but keeps it on disk.

Chapter summary

- Loop: edit, git status, git add, git commit. Check git status before and after.
- Staging lets you split unrelated work into separate, focused commits.
- Good messages: type, imperative summary, explain why.
- .gitignore keeps secrets, dependencies, and junk out of the repository.

CHECK YOURSELF

1. What does 'working tree clean' mean?
2. What is the difference between git diff and git diff --staged?
3. Write a good commit message for fixing a crash when a username is empty.
4. You committed .env by mistake and added it to .gitignore. Is it now ignored?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 5

Undoing mistakes safely

In this chapter you will
- pick the right undo command for each situation
- discard edits, unstage files, and fix the last commit
- undo commits with reset and revert
- know exactly which commands can lose work

You will need: the hello-git repository from Chapter 4

Beginners fear Git because they fear making a mistake. The truth: **anything you have committed is very hard to lose**. This chapter gives you a safe undo for every situation.

5.1 Which undo do I need?

Situation	Command	Danger?
I edited a file and want the old version back	git restore <file>	Edits are lost
I staged a file by mistake	git restore --staged <file>	Safe
Typo in my last commit message	git commit --amend -m "new"	Safe if not pushed
I forgot a file in my last commit	git add <file>; git commit --amend --no-edit	Safe if not pushed
Undo last commit, keep the changes staged	git reset --soft HEAD~1	Safe
Undo last commit, keep the changes unstaged	git reset HEAD~1	Safe
Undo last commit and throw the changes away	git reset --hard HEAD~1	**Loses uncommitted work**
Undo a commit that is already pushed	git revert <commit>	Safe (adds a new commit)
Delete untracked files	git clean -fd	**Permanent**

HANDS-ON LAB 5.1 Discard an edit and unstage a file

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ echo "oops, a mistake" >> hello.txt
$ cat hello.txt
Hello Git
Second line
oops, a mistake
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.txt

no changes added to commit (use "git add" and/or "git commit -a")
$ git restore hello.txt
$ cat hello.txt
Hello Git
Second line
```

> **WARNING** **`git restore <file>`** discards uncommitted edits **permanently**. There is no undo for work that was never committed.

Now stage something by accident and take it back:

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "draft" > draft.txt
$ git add draft.txt
$ git status -s
A  draft.txt
$ git restore --staged draft.txt
$ git status -s
?? draft.txt
$ rm draft.txt
```

HANDS-ON LAB 5.2 Fix the last commit

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "Third line" >> hello.txt
$ git commit -am "docs: ad third line"
[main 8e7d6c5] docs: ad third line
 1 file changed, 1 insertion(+)
$ git commit --amend -m "docs: add third line"
[main 4f3e2d1] docs: add third line
 Date: Mon Oct 12 09:05:00 2026 +0300
 1 file changed, 1 insertion(+)
$ git log --oneline -1
4f3e2d1 (HEAD -> main) docs: add third line
```

The commit got a **new hash** because amending replaces the commit. The **`-a`** in **`commit -am`** stages all modified tracked files for you (it does not include brand-new files).

> **WARNING** Only amend or reset commits that you have **not pushed**. Rewriting shared history confuses teammates (Chapter 8 explains why).

5.3 The three modes of reset

`git reset` moves the current branch back to an earlier commit. The mode decides what happens to your changes:

Mode	Branch pointer	Staging area	Your files	Use it to...
--soft	moves back	keeps changes staged	unchanged	redo the commit differently
--mixed (default)	moves back	cleared	unchanged	unstage and rethink
--hard	moves back	cleared	**overwritten**	throw everything away

`HEAD~1` means 'one commit before HEAD'. **`HEAD~3`** means three commits back.

HANDS-ON LAB 5.3 Reset soft, then hard

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "temp" > temp.txt
$ git add temp.txt
$ git commit -m "wip: temporary idea"
[main 1a1b1c1] wip: temporary idea
 1 file changed, 1 insertion(+)
 create mode 100644 temp.txt
$ git log --oneline
1a1b1c1 (HEAD -> main) wip: temporary idea
4f3e2d1 docs: add third line
```

```
...
$ git reset --soft HEAD~1
$ git status -s
A  temp.txt
$ git commit -m "wip: temporary idea"
[main 2b2c2d2] wip: temporary idea
 1 file changed, 1 insertion(+)
 create mode 100644 temp.txt
```

The soft reset removed the commit but kept the file staged, so we could recommit it. Now the dangerous one:

```
TERMINAL (Git Bash or macOS Terminal)

$ git reset --hard HEAD~1
HEAD is now at 4f3e2d1 docs: add third line
$ ls
hello.txt  readme.txt
```

The commit and its file are gone from your branch and your folder. But committed work is almost never truly lost: Chapter 12 shows how **`git reflog`** brings it back.

HANDS-ON LAB 5.4 Revert a commit safely

revert does not erase history. It adds a *new* commit that does the opposite of an earlier one, which is why it is safe for work you have already shared.

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "risky feature" > risky.txt
$ git add risky.txt && git commit -m "feat: add risky feature"
[main 3c3d3e3] feat: add risky feature
$ git revert --no-edit HEAD
[main 4d4e4f4] Revert "feat: add risky feature"
 1 file changed, 1 deletion(-)
 delete mode 100644 risky.txt
$ git log --oneline -3
4d4e4f4 (HEAD -> main) Revert "feat: add risky feature"
3c3d3e3 feat: add risky feature
4f3e2d1 docs: add third line
```

KEY IDEA **Not shared yet?** reset or amend. **Already shared?** revert. That one rule prevents most Git disasters.

Removing untracked files

```
TERMINAL (Git Bash or macOS Terminal)

$ touch junk1.txt junk2.txt
$ git clean -n                    # dry run: shows what WOULD be deleted
Would remove junk1.txt
Would remove junk2.txt
$ git clean -fd                   # really delete them
Removing junk1.txt
Removing junk2.txt
```

Chapter summary

- restore discards edits; restore --staged unstages; amend fixes the last commit.
- reset moves a branch back (soft keeps staged, mixed keeps files, hard discards).
- revert adds a new commit that undoes an old one; use it for shared history.
- Dangerous: reset --hard, restore, clean -fd. Run git status and git clean -n first.

CHECK YOURSELF

1. You staged a file by mistake. Which command unstages it and keeps your edits?
2. What is the difference between git reset --soft HEAD~1 and git reset --hard HEAD~1?
3. A bad commit is already pushed to GitHub. Reset or revert?
4. What does git clean -n do?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 6

Branching

In this chapter you will
- explain what a branch really is
- create, switch, rename, and delete branches
- read the commit graph with git log --graph
- recover from a detached HEAD

You will need: the hello-git repository

6.1 Why branches?

A **branch** is a separate line of development. You can try an idea, build a feature, or fix a bug without touching the stable code on **main**. When it works, you merge it back. When it does not, you delete the branch and nothing is harmed.

ANALOGY — Branches are **parallel universes** for your project. In one universe you are building the login page; in another you are fixing a bug. Nothing leaks between them until you decide to merge.

KEY IDEA — The standard habit: **one branch per task.** Keep main always working.

Command	What it does
`git branch`	List local branches. The current one has a * next to it.
`git branch -a`	List local and remote branches.
`git branch -vv`	List branches with last commit and remote tracking info.
`git branch name`	Create a branch (does not switch to it).
`git switch name`	Move onto an existing branch.
`git switch -c name`	Create a branch **and** switch to it.
`git switch -`	Jump back to the previous branch.
`git branch -m new-name`	Rename the current branch.
`git branch -d name`	Delete a merged branch (Git refuses if unmerged).
`git branch -D name`	Force-delete even if unmerged. Be sure!

NOTE — Older tutorials use **`git checkout <name>`** and **`git checkout -b <name>`**. They still work. **switch** (and **restore**) were added in Git 2.23 to split checkout's many jobs into clearer commands.

Naming branches

Prefix	Use for	Example
feature/	new functionality	feature/login-form
fix/	bug fixes	fix/cart-total
hotfix/	urgent production fixes	hotfix/payment-crash
docs/	documentation	docs/readme-setup
chore/	maintenance	chore/update-deps

Use lowercase and hyphens, avoid spaces, and keep names short but descriptive.

HANDS-ON LAB 6.1 Create a branch, commit on it, and watch files change

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ git branch
* main
$ git switch -c feature/greeting
Switched to a new branch 'feature/greeting'
$ echo "Habari" > greeting.txt
$ git add greeting.txt
$ git commit -m "feat: add greeting"
[feature/greeting 8a7b6c5] feat: add greeting
 1 file changed, 1 insertion(+)
 create mode 100644 greeting.txt
$ ls
greeting.txt  hello.txt  readme.txt
$ git switch main
Switched to branch 'main'
$ ls
hello.txt  readme.txt
```

greeting.txt vanished when you went back to main. It is not deleted; it lives on the other branch. Switch back and it returns:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch feature/greeting
Switched to branch 'feature/greeting'
$ ls
greeting.txt  hello.txt  readme.txt
$ git log --oneline --graph --all
* 8a7b6c5 (HEAD -> feature/greeting) feat: add greeting
* 4f3e2d1 (main) docs: add third line
* 7c1d4e2 docs: add second line to hello
* 3f2a9c1 feat: add hello file
```

> **PRO TIP** **`git log --oneline --graph --all`** is the best way to *see* your branches. Create an alias for it (Chapter 12): **`git config --global alias.lg "log --oneline --graph --all"`**, then just type **`git lg`**.

HANDS-ON LAB 6.2 Detached HEAD and how to escape

Checking out a commit instead of a branch puts you in **detached HEAD** state. It is not an error, but new commits there belong to no branch and can be lost.

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch --detach HEAD~1
HEAD is now at 4f3e2d1 docs: add third line
$ git status
HEAD detached at 4f3e2d1
nothing to commit, working tree clean
```

Two ways out:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch -c experiment        # keep work from here: give it a branch name
Switched to a new branch 'experiment'
$ git switch main                 # or just leave and abandon anything done
$ git branch -d experiment
Deleted branch experiment (was 4f3e2d1).
```

Chapter summary

- A branch is a movable pointer to a commit; creating one is instant and free.
- One branch per task. Use switch -c to create and move.
- git log --oneline --graph --all shows the shape of your history.
- Detached HEAD is safe to explore; create a branch to keep any commits.

CHECK YOURSELF

1. Which command creates a branch and switches to it in one step?
2. Why did greeting.txt disappear when you switched to main?
3. What command shows the branch structure as a graph?
4. You are in a detached HEAD and made a good commit. How do you keep it?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 7

Merging and resolving conflicts

In this chapter you will
- merge one branch into another
- tell a fast-forward merge from a merge commit
- resolve a merge conflict with confidence
- abort a merge that goes wrong

You will need: the hello-git repository, plus a new practice repo

7.1 What a merge does

Merging combines the work of two branches. You stand on the branch that should *receive* the work and say which branch to bring in:

```
THE PATTERN

$ git switch main              # stand on the receiving branch
$ git merge feature/greeting   # bring the feature in
```

Fast-forward

If main has not moved since you branched, Git simply slides the main pointer forward. No new commit is needed:

```
BEFORE                          AFTER (fast-forward)
A---B   main                    A---B---C---D   main, feature
     \
      C---D   feature
```

Three-way merge (merge commit)

If both branches have new commits, Git creates a **merge commit** that has two parents and joins the histories:

```
BEFORE                                  AFTER
          C---D   feature                       C---D
         /                                     /     \
A---B---E---F   main                  A---B---E---F---M   main
```

HANDS-ON LAB 7.1 A fast-forward merge

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ git switch main
Switched to branch 'main'
$ git merge feature/greeting
Updating 4f3e2d1..8a7b6c5
Fast-forward
 greeting.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 greeting.txt
$ git branch -d feature/greeting
Deleted branch feature/greeting (was 8a7b6c5).
```

Notice the word **Fast-forward** and that Git made no new commit. Branch deleted: the work now lives on main.

HANDS-ON LAB 7.2 A merge commit

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch -c feature/footer
Switched to a new branch 'feature/footer'
$ echo "Footer text" > footer.txt
$ git add footer.txt && git commit -m "feat: add footer"
[feature/footer 6f5e4d3] feat: add footer
$ git switch main
$ echo "Header text" > header.txt
$ git add header.txt && git commit -m "feat: add header"
[main 9a8b7c6] feat: add header
$ git merge feature/footer --no-edit
Merge made by the 'ort' strategy.
 footer.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 footer.txt
$ git log --oneline --graph
*   b7a6958 (HEAD -> main) Merge branch 'feature/footer'
|\
| * 6f5e4d3 (feature/footer) feat: add footer
* | 9a8b7c6 feat: add header
|/
* 8a7b6c5 feat: add greeting
```

This time Git had to create a merge commit (**b7a6958**) because both branches had moved. The graph shows the split and the join.

Command	What it does
`git merge --no-ff branch`	Always create a merge commit, even if a fast-forward is possible. Keeps a record that a feature branch existed.
`git merge --squash branch`	Combine all the branch's changes into one set of staged changes; you then make a single commit.
`git merge --abort`	Cancel a merge that has hit conflicts and return to how things were.

7.2 Merge conflicts

A conflict is **not an error**. It is Git saying: 'Two people changed the same lines, and I cannot decide whose version wins. Please choose.' It happens when both branches edit the same part of a file, or one deletes a file the other edited.

> **ANALOGY** Two people edit the same sentence of a shared document on different days. When their copies meet, someone has to decide what the final sentence should say. Git is just asking a human to be that someone.

HANDS-ON LAB 7.3 Create and resolve a conflict

We build a deliberate conflict in a fresh repository, so you can practise safely.

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book
$ mkdir conflict-lab && cd conflict-lab
$ git init
$ echo "Title: Our Site" > title.txt
$ git add . && git commit -m "feat: add title"
[main (root-commit) 1f2e3d4] feat: add title
 1 file changed, 1 insertion(+)
 create mode 100644 title.txt
```

Make a branch and change the title one way:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git switch -c awesome
Switched to a new branch 'awesome'
$ echo "Title: Our Awesome Site" > title.txt
$ git commit -am "feat: awesome title"
[awesome 5d4c3b2] feat: awesome title
```

Go back to main and change the **same line** a different way:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git switch main
Switched to branch 'main'
$ echo "Title: Our Amazing Site" > title.txt
$ git commit -am "feat: amazing title"
[main 9e8d7c6] feat: amazing title
```

Now merge. Git cannot choose:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git merge awesome
Auto-merging title.txt
CONFLICT (content): Merge conflict in title.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   title.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Open the file (**`code title.txt`**). Git has written **conflict markers** into it:

```
<<<<<<< HEAD
Title: Our Amazing Site          <-- the version on the branch you are on (main)
=======
Title: Our Awesome Site          <-- the version coming in (awesome)
>>>>>>> awesome
```

Marker	Meaning
`<<<<<<< HEAD`	Start of **your** version (the current branch)
`=======`	Divider between the two versions
`>>>>>>> awesome`	End of the **incoming** version (named after the other branch)

Resolve it in three steps: **(1)** decide what the final text should be, **(2)** delete all three marker lines, and **(3)** save. For this lab, make the file say:

```
Title: Our Awesome and Amazing Site
```

> **PRO TIP** In VS Code, conflicts appear with clickable buttons: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**. They simply edit the markers for you.

Tell Git it is resolved by staging the file, then commit:

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ git add title.txt
```

```
$ git commit -m "merge: combine title ideas"
[main a1b2c3d] merge: combine title ideas
$ git log --oneline --graph --all
*   a1b2c3d (HEAD -> main) merge: combine title ideas
|\
| * 5d4c3b2 (awesome) feat: awesome title
* | 9e8d7c6 feat: amazing title
|/
* 1f2e3d4 feat: add title
```

KEY IDEA Resolve: **edit, remove markers, git add, git commit**. Cannot cope? **`git merge --abort`** puts everything back as it was.

HANDS-ON LAB 7.4 Abort a merge and take one side wholesale

```
TERMINAL (Git Bash or macOS Terminal)

$ git merge awesome               # (imagine another conflict happened)
CONFLICT (content): Merge conflict in title.txt
$ git merge --abort               # changed my mind: back to before the merge
$ git status
On branch main
nothing to commit, working tree clean
```

When you simply want **one side's** whole file, without editing:

```
TERMINAL (Git Bash or macOS Terminal)

$ git checkout --ours title.txt     # keep the version from the branch you are on
$ git checkout --theirs title.txt   # keep the version from the incoming branch
$ git add title.txt
```

How to have fewer conflicts

- Pull and merge from main **often**; small, frequent merges mean small, easy conflicts.
- Keep branches **short-lived** and tasks small.
- Talk to teammates before big changes to the same file, and avoid mass reformatting mixed with real changes.

Chapter summary

• Stand on the receiving branch and run git merge <other>.
• Fast-forward = pointer slides. Merge commit = two parents joining the histories.
• Conflicts are normal: open the file, choose the final text, remove markers, git add, git commit.
• git merge --abort is your safety exit.

CHECK YOURSELF

1. You want feature to be merged into main. On which branch do you run git merge feature?
2. When does Git do a fast-forward merge?
3. What do the lines <<<<<<<, =======, and >>>>>>> mean in a conflicted file?
4. You are halfway through a conflicting merge and want out. Which command?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 8

Rebasing and tidying history

In this chapter you will
- understand what rebase does and how it differs from merge
- know the golden rule of rebasing
- squash messy commits with interactive rebase
- continue or abort a rebase that hits a conflict

You will need: a new practice repo; Chapters 6 and 7 done

8.1 Rebase: replaying your work

Rebase takes your branch's commits and **replays** them on top of another branch, as if you had started your work from the latest commit. The result is a straight, tidy history instead of a web of merges.

```
BEFORE                                AFTER  git rebase main  (while on feature)

          A---B   feature                     D---E---F---G   main
         /                                                  \
D---E---F---G   main                                         A'--B'   feature
```

Notice A' and B': the commits are **rewritten with new hashes**, because their parent changed. That is why rebase must be used carefully.

KEY IDEA

Golden rule: never rebase commits that other people already have (public or shared branches). Rebase *your own* local work; merge shared work.

	Merge	Rebase
History	Records exactly what happened, including branching	Rewrites commits into a clean straight line
Safe on shared branches?	Yes	**No**
Typical use	Bringing a finished feature into main	Updating your own branch with the latest main; tidying commits before a pull request
Downside	A busier graph	Rewrites hashes; dangerous if misused

HANDS-ON LAB 8.1 Rebase a feature branch onto main

TERMINAL (Git Bash or macOS Terminal)

```
$ cd ~/git-book
$ mkdir rebase-lab && cd rebase-lab
$ git init
$ echo "base" > app.txt && git add . && git commit -m "chore: base"
[main (root-commit) 2a1b0c9] chore: base
$ git switch -c feature/search
$ echo "search v1" > search.txt && git add . && git commit -m "feat: add search"
[feature/search 6d5e4f3] feat: add search
$ git switch main
$ echo "news" > news.txt && git add . && git commit -m "feat: add news"
[main 4b3a2c1] feat: add news
$ git log --oneline --graph --all
* 4b3a2c1 (HEAD -> main) feat: add news
| * 6d5e4f3 (feature/search) feat: add search
|/
```

```
* 2a1b0c9 chore: base
```

The branches have diverged. Rebase the feature on top of main:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch feature/search
Switched to branch 'feature/search'
$ git rebase main
Successfully rebased and updated refs/heads/feature/search.
$ git log --oneline --graph --all
* 8f7e6d5 (HEAD -> feature/search) feat: add search
* 4b3a2c1 (main) feat: add news
* 2a1b0c9 chore: base
```

The history is now a straight line, and the search commit has a **new hash** (8f7e6d5, not 6d5e4f3). Finish by merging into main, which is now a clean fast-forward:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch main
$ git merge feature/search
Updating 4b3a2c1..8f7e6d5
Fast-forward
 search.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 search.txt
```

8.2 Interactive rebase: tidy before you share

You will often make small fix-up commits like 'fix typo' while working. Before opening a pull request you can **squash** them into one tidy commit with an **interactive rebase**.

HANDS-ON LAB 8.2 Squash three commits into one

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch -c feature/header
$ echo "header" > header.txt && git add . && git commit -m "feat: add header"
$ echo "more" >> header.txt && git commit -am "fix typo"
$ echo "again" >> header.txt && git commit -am "fix typo again"
$ git log --oneline
3333333 (HEAD -> feature/header) fix typo again
2222222 fix typo
1111111 feat: add header
8f7e6d5 feat: add search
...
```

Start an interactive rebase over the last three commits:

```
TERMINAL (Git Bash or macOS Terminal)

$ git rebase -i HEAD~3
```

Your editor opens with a todo list (oldest commit first). Change the word at the start of the last two lines from **pick** to **fixup**:

```
THE EDITOR SHOWS A TODO LIST

pick 1111111 feat: add header
fixup 2222222 fix typo
fixup 3333333 fix typo again

# Commands:
# p, pick   = use commit
```

```
# r, reword = use commit, but edit the commit message
# s, squash = use commit, but meld into previous commit (and combine messages)
# f, fixup  = like squash, but discard this commit's log message
# d, drop   = remove commit
```

Save and close the editor. Git replays the commits and melds them:

```
TERMINAL (Git Bash or macOS Terminal)

$ git log --oneline
9a9b9c9 (HEAD -> feature/header) feat: add header
8f7e6d5 feat: add search
...
```

Three messy commits became one clean one. Use **reword** to rename a commit, **squash** to combine messages, **drop** to delete a commit, and reorder lines to reorder commits.

PRO TIP Abandon an interactive rebase at any time with **`git rebase --abort`**. Nothing is changed until you save and finish.

8.3 When a rebase hits a conflict

Rebase replays commits one at a time, so a conflict can appear mid-way. The steps are almost the same as for a merge:

1. Git stops and tells you which file conflicts. Open it and fix the markers exactly as in Chapter 7.
2. Stage the file: **`git add <file>`**.
3. Continue: **`git rebase --continue`**. (Do **not** run git commit.)
4. Repeat until finished. At any point: **`git rebase --abort`** returns to how things were, and **`git rebase --skip`** drops the troublesome commit.

Chapter summary

• Rebase replays your commits on top of another branch for a linear history; commits get new hashes.
• Golden rule: never rebase commits others already have.
• git rebase -i HEAD~N lets you squash, reword, reorder, and drop your own recent commits.
• Conflict during rebase: fix, git add, git rebase --continue (or --abort).

CHECK YOURSELF

1. Why do rebased commits get new hashes?
2. State the golden rule of rebasing.
3. In an interactive rebase, which word turns a commit into part of the previous one and discards its message?
4. You hit a conflict during a rebase and fixed the file. What two commands come next?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 9

GitHub and remote repositories

In this chapter you will
- create a GitHub account and secure it
- connect your computer to GitHub with an SSH key
- push a local project, then clone, fetch, and pull
- understand why a push is sometimes rejected, and fix it

You will need: the hello-git repository from earlier chapters and a free GitHub account

9.1 Create and secure your account

1. Go to **github.com** and click **Sign up**. Use an email address you will keep for years.
2. Choose a professional username (it appears in your URL and on every contribution), for example **firstname-lastname**.
3. Verify your email address.
4. Turn on **two-factor authentication**: Settings, then *Password and authentication*, then *Enable two-factor authentication* (use an authenticator app).
5. Make sure the email in `git config user.email` matches an email on your account so commits link to your profile.

PRO TIP Your GitHub profile is part of your CV. Add a photo, a short bio, and your location. Later you will pin your best repositories to it (Chapter 14).

9.2 Proving who you are

To push to GitHub you must authenticate. Since August 2021 GitHub no longer accepts your account password for Git commands. You have three good options:

Method	How it works	Best for
SSH key (recommended)	You make a key pair. The private key stays on your computer; you upload the public key to GitHub. No prompts afterwards.	Daily work
HTTPS with a token	Use a Personal Access Token as the password (or let the credential manager handle sign-in)	Quick start; restricted networks
GitHub CLI (gh auth login)	A guided sign-in that sets up credentials for you	Simplest setup

HANDS-ON LAB 9.1 Set up an SSH key

```
TERMINAL (Git Bash or macOS Terminal)

$ ssh-keygen -t ed25519 -C "you@example.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/student/.ssh/id_ed25519):   # press Enter
Enter passphrase (empty for no passphrase):                               # choose one
Your identification has been saved in /c/Users/student/.ssh/id_ed25519
Your public key has been saved in /c/Users/student/.ssh/id_ed25519.pub
```

This created two files. The **private** key (`id_ed25519`) must **never** be shared. The **public** key (`.pub`) is safe to upload. Start the key agent and add your key:

WINDOWS (Git Bash)	macOS (Terminal)

`eval "$(ssh-agent -s)"`
`ssh-add ~/.ssh/id_ed25519`
 	
Copy the public key to your clipboard:
`clip < ~/.ssh/id_ed25519.pub`
	`eval "$(ssh-agent -s)"`
	`ssh-add --apple-use-keychain ~/.ssh/id_ed25519`
	
	Copy the public key to your clipboard:
	`pbcopy < ~/.ssh/id_ed25519.pub`

Now add it to GitHub: **Settings, then SSH and GPG keys, then New SSH key**. Give it a title (for example 'My laptop'), paste the key, and save. Test the connection:

```
TERMINAL (Git Bash or macOS Terminal)

$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.x.x)' can't be established.
Are you sure you want to continue connecting (yes/no)? yes
Hi your-username! You've successfully authenticated, but GitHub does not provide shell access.
```

That 'Hi your-username!' message means success. (The first-time question is normal; type **yes**.)

NOTE Prefer a simpler setup? Install the GitHub CLI (`winget install --id GitHub.cli` on Windows, `brew install gh` on macOS) and run `gh auth login`. Follow the prompts. It configures authentication for you.

9.3 Remotes: the vocabulary

Command	What it does
`git remote -v`	List your remotes and their URLs.
`git remote add origin <url>`	Connect this repository to a remote and call it **origin**.
`git push -u origin main`	Upload main to origin. The **-u** links them so later you can type just **git push**.
`git clone <url>`	Download a complete copy of a remote repository and set up origin automatically.
`git fetch`	Download new commits from the remote but **do not change your files**. Safe any time.
`git pull`	**fetch + merge**: download and immediately integrate remote changes into your current branch.
`git push`	Upload your local commits to the tracked remote branch.

KEY IDEA **fetch looks. pull looks and combines.** When unsure, run `git fetch` first, check what arrived, then decide.

origin is just the default nickname for the remote you cloned from or first connected. A **tracking branch** is a local branch linked to a remote one (like main tracking origin/main), so Git knows where push and pull should go.

HANDS-ON LAB 9.2 Create a GitHub repository and push your project

In the browser: click the **+** at the top right of GitHub, then **New repository**. Name it **hello-git**, keep it Public, and **leave 'Add a README' unchecked** (your local project already has history). Click **Create repository**. GitHub shows a page with commands; use your own URL:

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ git remote add origin git@github.com:YOUR-USERNAME/hello-git.git
$ git remote -v
origin  git@github.com:YOUR-USERNAME/hello-git.git (fetch)
origin  git@github.com:YOUR-USERNAME/hello-git.git (push)
$ git branch -M main
$ git push -u origin main
```

```
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (18/18), 1.54 KiB | 1.54 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0), pack-reused 0
To github.com:YOUR-USERNAME/hello-git.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Refresh the repository page on GitHub. Your files and the whole commit history are there. **Congratulations: your first push.**

HANDS-ON LAB 9.3 Fetch and pull changes made elsewhere

To simulate a teammate, edit a file **on GitHub**: open **hello.txt**, click the pencil icon, add a line, and click **Commit changes**. Your local copy does not know yet:

```
TERMINAL (Git Bash or macOS Terminal)

$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
Unpacking objects: 100% (3/3), done.
From github.com:YOUR-USERNAME/hello-git
   4f3e2d1..a9b8c7d  main       -> origin/main
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
$ git log --oneline origin/main -1
a9b8c7d Update hello.txt
$ git pull
Updating 4f3e2d1..a9b8c7d
Fast-forward
 hello.txt | 1 +
 1 file changed, 1 insertion(+)
```

fetch brought the commit into **`origin/main`** (your local picture of the remote) without touching your files. **pull** then merged it into main.

HANDS-ON LAB 9.4 Clone, and meet a rejected push

Work as **two people**: your original folder is Person A; a fresh clone is Person B.

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book
$ git clone git@github.com:YOUR-USERNAME/hello-git.git teammate
Cloning into 'teammate'...
remote: Enumerating objects: 21, done.
Receiving objects: 100% (21/21), done.
$ cd teammate
$ echo "Person B was here" >> hello.txt
$ git commit -am "docs: Person B adds a line"
$ git push
To github.com:YOUR-USERNAME/hello-git.git
   a9b8c7d..c0d1e2f  main -> main
```

Now, as Person A (the original folder), make a different commit and try to push without pulling first:

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ echo "Person A was here" >> readme.txt
```

```
$ git commit -am "docs: Person A adds a line"
$ git push
To github.com:YOUR-USERNAME/hello-git.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:YOUR-USERNAME/hello-git.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
```

This is the most common Git message a beginner meets. It is not a disaster. It means: 'GitHub has commits you do not, so I will not overwrite them.' Fix it by integrating first:

TERMINAL (Git Bash or macOS Terminal)

```
$ git pull --rebase
Successfully rebased and updated refs/heads/main.
$ git push
To github.com:YOUR-USERNAME/hello-git.git
   c0d1e2f..d3e4f5a  main -> main
```

NOTE **`git pull`** (merge) works too and creates a merge commit; **`git pull --rebase`** keeps history in a straight line. Teams choose one style. If Git warns *You have divergent branches*, set your choice once: **`git config --global pull.rebase true`** (or **`false`** for merge).

WARNING The temptation is to run **`git push --force`** to 'make the error go away'. **Do not.** It overwrites your teammate's work on GitHub. The safer variant, **`--force-with-lease`**, is only for your own branch after a rebase.

9.4 Your README: the front page

GitHub shows a file named **README.md** on your repository's front page. Write it in **Markdown**, a simple text format:

MARKDOWN CHEAT SHEET

````
# Project title                 (one # = biggest heading)
## A section                    (two # = smaller heading)

A paragraph with **bold**, *italic* and `inline code`.

- a bullet point
- another bullet

1. a numbered step
2. another step

[link text](https://example.com)
![alt text for an image](screenshot.png)

```bash
git clone git@github.com:you/project.git
```
````

A good README answers: **What is it? How do I install it? How do I run it? How do I contribute?**

Chapter summary

- Create a GitHub account with 2FA, and authenticate with an SSH key (or gh auth login).
- remote add, push -u origin main, clone, fetch (look), pull (look and combine).
- A rejected push means the remote has work you lack: pull (or pull --rebase), then push.
- Never force-push to shared branches. README.md is your project's front page.

CHECK YOURSELF

1. What is the difference between git fetch and git pull?
2. What does 'origin' mean?
3. Your push is rejected with 'fetch first'. What should you do?
4. Which key file must you never share, the .pub or the one without it?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 10

Collaborating: issues, forks, and pull requests

In this chapter you will
• track work with issues
• open, review, and merge a pull request
• contribute to a project you do not own using a fork
• keep your fork in sync and resolve conflicts in a PR

You will need: your hello-git repository on GitHub

10.1 Issues: tracking the work

An **issue** is a task, bug report, or idea attached to a repository. Each has a number (#12), a discussion thread, labels (bug, enhancement, good first issue), and optional assignees and milestones. Write issues that others can act on:

- A **clear title**: 'Login button does nothing on mobile Safari', not 'bug'.
- **Steps to reproduce**, what you expected, and what happened (add screenshots).
- Environment details: browser, operating system, version.

KEY IDEA — Link work to issues. Writing `Closes #12` in a pull request or commit message **automatically closes issue 12** when the PR is merged. Also works with Fixes and Resolves.

10.2 The pull request (PR)

A **pull request** is a proposal to merge one branch into another, with a space for discussion, review, and automated checks. It is the heart of collaboration on GitHub.

ANALOGY — A pull request is submitting a draft to an editor. You are not saying 'publish this now'; you are saying 'please read this, tell me what to improve, and then we decide together'.

```
THE PULL REQUEST LIFECYCLE

 branch --> commit --> push --> open PR --> review --> fix --> approve --> merge --> delete
```

What makes a good PR

- **Small and focused.** One purpose per PR. Reviewers cannot review 3,000 lines well.
- **A clear title and description:** what changed, why, how to test it, screenshots for UI changes.
- **Linked to an issue** (Closes #12) and with passing automated checks.
- **Self-reviewed first.** Read your own diff on GitHub before asking others.

HANDS-ON LAB 10.1 Open and merge a pull request (solo practice)

You can practise the whole flow on your own repository, so nothing is at risk.

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ git switch main && git pull
$ git switch -c feature/about-page
Switched to a new branch 'feature/about-page'
$ echo "About this project" > about.txt
$ git add about.txt
$ git commit -m "docs: add about page"
[feature/about-page 5a6b7c8] docs: add about page
```

```
$ git push -u origin feature/about-page
remote: Create a pull request for 'feature/about-page' on GitHub by visiting:
remote:      https://github.com/YOUR-USERNAME/hello-git/pull/new/feature/about-page
To github.com:YOUR-USERNAME/hello-git.git
 * [new branch]      feature/about-page -> feature/about-page
branch 'feature/about-page' set up to track 'origin/feature/about-page'.
```

1. Open the link GitHub printed (or click the yellow **Compare and pull request** banner on the repository page).
2. Write a title such as **docs: add about page** and a short description. Check the **Files changed** tab: this is your diff.
3. Click **Create pull request**.
4. Click **Merge pull request** (choose **Squash and merge**), confirm, then click **Delete branch**.

Now bring the merged result back to your computer and clean up:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch main
$ git pull
Updating a9b8c7d..e1f2a3b
Fast-forward
 about.txt | 1 +
$ git branch -d feature/about-page
warning: deleting branch 'feature/about-page' that has been merged to
         'refs/remotes/origin/feature/about-page', but not yet merged to HEAD.
Deleted branch feature/about-page (was 5a6b7c8).
$ git fetch --prune                  # forget remote branches deleted on GitHub
```

The three ways to merge a PR

Option	Result	Use when
Create a merge commit	All commits kept, plus a merge commit	You want a full record of the branch
Squash and merge	All the PR's commits become **one** commit on main	Messy WIP commits; you want clean history (very common)
Rebase and merge	Commits replayed onto main, no merge commit	You want a linear history and clean individual commits

10.3 Code review

A review is a conversation, not a judgement. On the **Files changed** tab you can click the **+** beside any line to comment, or use a **suggestion** block that the author can apply with one click. A review ends with one of three outcomes: **Comment**, **Approve**, or **Request changes**.

As a reviewer	As an author
Review the code, not the person.	Keep PRs small so review is quick.
Be specific and kind: 'What happens if the list is empty?'	Say what you want feedback on.
Separate must-fix from optional ('nit: rename this?').	Do not take comments personally; thank reviewers.
Praise good work. Approve promptly when ready.	Reply to every comment, then re-request review.

HANDS-ON LAB 10.2 Review practice with a partner

1. Pair up. Each person opens a small PR on their own repository (add a line to the README, for example).
2. Add your partner as a collaborator: repository **Settings, Collaborators, Add people**.
3. Review your partner's PR: leave one line comment, one **suggestion**, then **Approve**.
4. Respond to the review on your own PR, apply the suggestion, and merge.

10.4 Contributing to a project you do not own

You cannot push to other people's repositories. The open-source solution is the **fork**: your own copy of someone else's repository on GitHub. You work in your fork and open a pull request back to the original.

```
THE FORK AND PULL REQUEST FLOW

  original repo (upstream)  ---fork--->  your fork (origin)  ---clone--->  your computer
          ^                                  ^                              |
          |                                  |                        branch, commit
          +------ pull request <---------- git push <-----------------------+
```

Name	Meaning
upstream	The original repository you forked from (you add this remote yourself)
origin	Your fork on GitHub (set automatically when you clone it)

HANDS-ON LAB 10.3 Make a real open-source contribution

The community project **firstcontributions/first-contributions** exists exactly for this practice. Open its page on GitHub and follow these steps:

```
TERMINAL (Git Bash or macOS Terminal)

# 1) On GitHub: click Fork on the repository page.
# 2) Clone YOUR fork:
$ cd ~/git-book
$ git clone git@github.com:YOUR-USERNAME/first-contributions.git
$ cd first-contributions
# 3) Add the original as upstream:
$ git remote add upstream https://github.com/firstcontributions/first-contributions.git
$ git remote -v
origin    git@github.com:YOUR-USERNAME/first-contributions.git (fetch)
origin    git@github.com:YOUR-USERNAME/first-contributions.git (push)
upstream  https://github.com/firstcontributions/first-contributions.git (fetch)
upstream  https://github.com/firstcontributions/first-contributions.git (push)
# 4) Create a branch, make the change the project's README asks for, commit:
$ git switch -c add-your-name
$ git add .
$ git commit -m "docs: add YOUR NAME to contributors"
# 5) Push and open the pull request:
$ git push -u origin add-your-name
```

On GitHub, click **Compare and pull request** and submit. When a maintainer merges it, you are officially an open-source contributor, and it appears on your profile.

Keeping your fork up to date

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch main
$ git fetch upstream
From https://github.com/firstcontributions/first-contributions
 * [new branch]      main       -> upstream/main
$ git merge upstream/main          # or: git rebase upstream/main
```

```
$ git push origin main
```

Conflicts inside a pull request

If GitHub says *This branch has conflicts that must be resolved*, main moved while you worked. Fix it locally:

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch feature/about-page
$ git fetch origin
$ git merge origin/main              # or: git rebase origin/main
# resolve the conflict markers, then:
$ git add <file>
$ git commit                         # (for rebase: git rebase --continue)
$ git push                           # (after a rebase: git push --force-with-lease)
```

Protecting your main branch

On a team, main should be protected. In **Settings, Branches (or Rulesets)** you can require: a pull request before merging, a number of approvals, passing status checks (your automated tests), up-to-date branches, and block force pushes. A file called **CODEOWNERS** can automatically request review from the right people. A **PR template** in **`.github/PULL_REQUEST_TEMPLATE.md`** gives every PR a checklist.

Chapter summary

- Issues track work; 'Closes #N' links and auto-closes them.
- A PR is a proposal with review: small, well described, linked, passing checks.
- Squash and merge keeps history tidy; delete branches after merging.
- Fork, clone, add upstream, branch, push, PR: the open-source flow.

CHECK YOURSELF

1. What does writing 'Closes #12' in a PR description do?
2. Why do open-source contributors fork instead of pushing to the original repository?
3. What is the difference between origin and upstream?
4. Name the three ways to merge a pull request on GitHub.

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 11

Team workflows and conventions

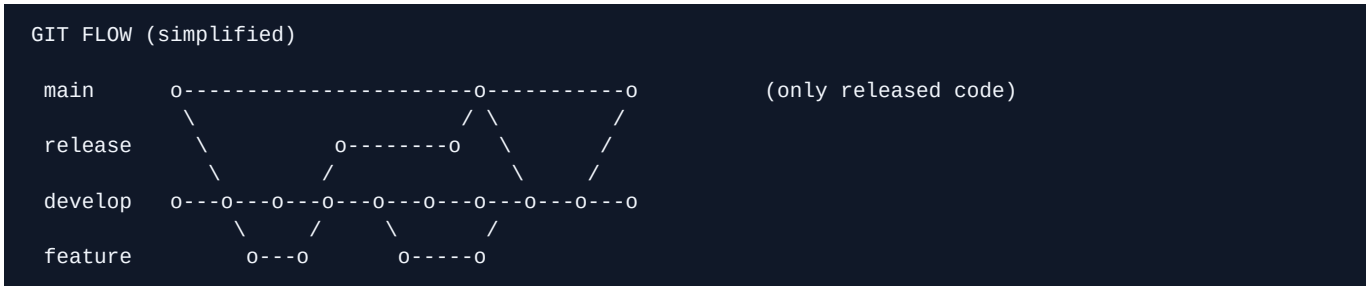
In this chapter you will
- compare GitHub Flow, Git Flow, and trunk-based development
- write Conventional Commits and understand semantic versioning
- tag a release and publish it on GitHub
- choose a sensible workflow for a team

You will need: your hello-git repository on GitHub

A team needs shared rules, or five developers will produce five different habits. A **workflow** is the set of traffic rules for branches and merging.

11.1 Three popular workflows

	GitHub Flow	Git Flow	Trunk-based
Idea	Branch off main, open a PR, merge, deploy	Several long-lived branches with strict roles	Everyone merges into one main branch, often daily
Branches	main + short feature branches	main, develop, feature/*, release/*, hotfix/*	main + very short-lived branches
Releases	Deploy from main after merge	Planned versioned releases via release branches	Continuous, often behind feature flags
Best for	Most web teams and open source	Scheduled releases, several supported versions	Fast teams with strong automated tests
Watch out	Needs reliable tests	Long-lived branches drift and merge painfully	Main must always work; needs discipline



KEY IDEA There is no best workflow. The best one is the one your team **will actually follow**, matched to how often you release and how good your tests are.

The routine behind almost every workflow

THE FEATURE-BRANCH ROUTINE

```
$ git switch main && git pull              # start from the latest main
$ git switch -c feature/login-form         # one branch per task
# ...edit, test...
$ git add -p && git commit -m "feat(auth): add login form"
$ git push -u origin feature/login-form
# open a pull request, get a review, fix feedback, merge on GitHub, then:
$ git switch main && git pull
$ git branch -d feature/login-form
```

11.2 Conventional Commits

A shared message format makes history readable and enables automation (changelogs, version bumps). The format is `type(scope): summary`.

Type	Use for	Example
feat	a new feature	feat: add password reset
fix	a bug fix	fix(cart): handle empty cart
docs	documentation only	docs: update setup steps
style	formatting, no behaviour change	style: format with prettier
refactor	restructure without changing behaviour	refactor: split auth service
test	add or fix tests	test: cover login errors
perf	performance improvement	perf: cache product queries
build / ci	build system or CI changes	ci: add test workflow
chore	routine maintenance	chore: update .gitignore
feat!	a **breaking change** (the ! flags it)	feat!: drop Node 16 support

11.3 Tags, versions, and releases

A **tag** is a permanent, named label on one commit, usually marking a release. **Semantic versioning** numbers releases as **MAJOR.MINOR.PATCH**:

Part	Bump it when...	Example
MAJOR	you make incompatible (breaking) changes	1.4.2 to **2**.0.0
MINOR	you add features in a backwards-compatible way	1.4.2 to 1.**5**.0
PATCH	you make backwards-compatible bug fixes	1.4.2 to 1.4.**3**

HANDS-ON LAB 11.1 Tag a release and publish it

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ cd ~/git-book/hello-git
$ git switch main && git pull
$ git tag -a v1.0.0 -m "First release"
$ git tag
v1.0.0
$ git show v1.0.0 --stat
tag v1.0.0
Tagger: Your Name <you@example.com>
Date:   Mon Oct 12 10:40:00 2026 +0300

First release

commit e1f2a3b... (HEAD -> main, tag: v1.0.0, origin/main)
$ git push origin v1.0.0
To github.com:YOUR-USERNAME/hello-git.git
 * [new tag]         v1.0.0 -> v1.0.0
```

On GitHub, open the repository, click **Releases** (right sidebar), then **Draft a new release**, choose the tag **v1.0.0**, click **Generate release notes**, and **Publish release**. GitHub lists the changes and offers a downloadable zip of the code at that version.

Command	What it does
`git tag`	List all tags.
`git tag -a v1.0.0 -m "msg"`	Create an annotated tag (recommended for releases).
`git push origin v1.0.0`	Push one tag (tags are not pushed by default).
`git push origin --tags`	Push all tags.
`git tag -d v1.0.0`	Delete a local tag.
`git switch --detach v1.0.0`	Look at the project exactly as it was at that release.

Chapter summary

- GitHub Flow suits most teams; Git Flow suits planned releases; trunk-based suits fast, well-tested teams.
- Conventional Commits (feat, fix, docs...) make history readable and automatable.
- Semantic versioning: MAJOR.MINOR.PATCH. Tag releases and publish them on GitHub.

CHECK YOURSELF

1. Which workflow uses main, develop, feature, release, and hotfix branches?
2. Write a Conventional Commit for adding a dark mode feature.
3. Your project is at 2.3.4. You fix a small bug without changing behaviour. What is the next version?
4. Are tags pushed to GitHub automatically with git push?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 12

Advanced tools

In this chapter you will
- shelve unfinished work with stash
- copy a single commit with cherry-pick
- rescue lost commits with reflog
- investigate history with blame, log search, and bisect
- speed up your work with aliases

You will need: a new practice repo; Chapters 4 to 9 done

12.1 Stash: the interruption drawer

You are halfway through a change and need to switch tasks right now, but you are not ready to commit. **Stash** shelves your uncommitted work and gives you a clean folder; you bring it back later.

ANALOGY

You are cooking when the phone rings. Stash puts the pan in the drawer so you can answer, and later you take it out exactly as it was.

HANDS-ON LAB 12.1 Stash and restore your work

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ echo "half-done work" >> hello.txt
$ git status -s
 M hello.txt
$ git stash push -m "wip: half-done idea"
Saved working directory and index state On main: wip: half-done idea
$ git status -s                          # clean again
$ git stash list
stash@{0}: On main: wip: half-done idea
$ git stash pop
On branch main
Changes not staged for commit:
        modified:   hello.txt
Dropped refs/stash@{0} (3b2a1c0...)
```

Command	What it does
`git stash push -m "msg"`	Shelve tracked changes with a label.
`git stash -u`	Also stash untracked (new) files.
`git stash list`	Show all stashes (stash@{0} is the newest).
`git stash show -p stash@{0}`	Show the changes inside a stash.
`git stash pop`	Re-apply the newest stash and remove it from the list.
`git stash apply`	Re-apply but keep it in the list.
`git stash drop stash@{0}`	Delete one stash.
`git stash clear`	Delete all stashes. Careful!

12.2 Cherry-pick: copy one commit

cherry-pick copies *one specific commit* from another branch onto your current branch, which is perfect for applying an urgent fix to a release branch without merging everything else.

HANDS-ON LAB 12.2 Cherry-pick a hotfix

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book && mkdir advanced-lab && cd advanced-lab
$ git init
$ echo "v1" > app.txt && git add . && git commit -m "chore: v1"
[main (root-commit) 1111aaa] chore: v1
$ git branch release                    # a release branch frozen at v1
$ echo "fix" > fix.txt && git add . && git commit -m "fix: critical bug"
[main 7e6d5c4] fix: critical bug
$ git switch release
Switched to branch 'release'
$ git cherry-pick 7e6d5c4
[release 3a2b1c0] fix: critical bug
 Date: Mon Oct 12 11:00:00 2026 +0300
 1 file changed, 1 insertion(+)
 create mode 100644 fix.txt
$ git log --oneline
3a2b1c0 (HEAD -> release) fix: critical bug
1111aaa chore: v1
```

The fix now exists on **release** as a *new commit with a new hash* (3a2b1c0). Hit a conflict? Fix it, **`git add`**, then **`git cherry-pick --continue`** (or **`--abort`**).

12.3 Reflog: the black-box recorder

Git secretly keeps a private log of **everywhere HEAD has been**, called the **reflog**. Even after a hard reset or a deleted branch, the commits are still stored for roughly 90 days, and the reflog tells you their hashes.

KEY IDEA Committed work is almost never lost. If you panic, run **`git reflog`**. (Uncommitted edits destroyed by reset --hard or restore are the exception, so commit early and often.)

HANDS-ON LAB 12.3 Delete commits on purpose, then rescue them

```
TERMINAL (Git Bash or macOS Terminal)

$ git switch main
$ for i in 2 3 4; do echo "line $i" >> app.txt; git commit -qam "change $i"; done
$ git log --oneline
b4b4b44 (HEAD -> main) change 4
a3a3a33 change 3
c2c2c22 change 2
7e6d5c4 fix: critical bug
1111aaa chore: v1
$ git reset --hard HEAD~2               # oh no! two commits vanish
HEAD is now at c2c2c22 change 2
$ git log --oneline -1
c2c2c22 (HEAD -> main) change 2
```

They look gone. Ask the reflog:

```
TERMINAL (Git Bash or macOS Terminal)

$ git reflog
c2c2c22 (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
b4b4b44 HEAD@{1}: commit: change 4
a3a3a33 HEAD@{2}: commit: change 3
c2c2c22 HEAD@{3}: commit: change 2
```

```
$ git branch rescue b4b4b44          # a safe way to recover: label the lost commit
$ git log --oneline rescue -1
b4b4b44 (rescue) change 4
$ git reset --hard b4b4b44           # or move main straight back
HEAD is now at b4b4b44 change 4
```

PRO TIP Recovering a deleted branch works the same way: **`git reflog`**, find the last commit of that branch, then **`git branch <name> <hash>`**.

12.4 Investigating history

Command	What it does
`git blame file`	Show who last changed each line of a file, and in which commit.
`git log -p -- file`	Show the full history of one file, with the changes.
`git log --grep="login"`	Find commits whose *message* contains 'login'.
`git log -S"password"`	Find commits that added or removed the text 'password' (the 'pickaxe').
`git log --author="Amina"`	Commits by one author.
`git log --since="2 weeks ago"`	Commits from a time range.
`git show <commit>`	One commit in detail: message and changes.
`git shortlog -sn`	Commits per author, as a leaderboard.

```
TERMINAL (Git Bash or macOS Terminal)

$ git blame hello.txt
3f2a9c1e (Your Name 2026-10-12 08:30:00 +0300 1) Hello Git
7c1d4e2f (Your Name 2026-10-12 08:40:00 +0300 2) Second line
```

Bisect: find the commit that broke it

Something worked last week and is broken now, across 200 commits. **git bisect** performs a binary search: it checks out the middle commit, you say good or bad, and it halves the range until it finds the culprit, in about 8 steps for 200 commits. It can even run a test for you.

HANDS-ON LAB 12.4 Find a bug with bisect

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book && mkdir bisect-lab && cd bisect-lab && git init
$ for i in 1 2 3 4 5 6 7 8 9 10; do
>   if [ $i -ge 6 ]; then echo "BUG $i" > status.txt; else echo "ok $i" > status.txt; fi
>   git add status.txt; git commit -qm "change $i"
> done
$ cat status.txt
BUG 10
```

Commit 6 introduced the bug. Let Git find it automatically, using a test that passes when the file does **not** contain BUG:

```
TERMINAL (Git Bash or macOS Terminal)

$ git bisect start HEAD HEAD~9           # bad = now, good = 9 commits ago
$ git bisect run sh -c '! grep -q BUG status.txt'
running  'sh' '-c' '! grep -q BUG status.txt'
...
6f5e4d3a2b1c0f9e8d7c6b5a4f3e2d1c0b9a8f7e is the first bad commit
commit 6f5e4d3a2b1c0f9e8d7c6b5a4f3e2d1c0b9a8f7e
    change 6
bisect found first bad commit
```

```
$ git bisect reset                    # return to where you started
```

> **NOTE** Your hashes will differ, but the message **change 6** will be identified as the first bad commit. In real life the 'test' is your project's test command.

12.5 Aliases: shortcuts you will use daily

```
TERMINAL (Git Bash or macOS Terminal)

$ git config --global alias.st "status -s"
$ git config --global alias.lg "log --oneline --graph --all --decorate"
$ git config --global alias.last "log -1 HEAD"
$ git config --global alias.unstage "restore --staged"
$ git st                      # now just 'git st' instead of 'git status -s'
$ git lg                      # the commit graph in two letters
```

12.6 A few more power tools

Command	What it does
`git worktree add ../fix fix-branch`	Check out another branch in a second folder so you can work on two branches at once with no stashing.
`git commit --fixup <commit>`	Create a fix-up commit for an earlier one; later **git rebase -i --autosquash** folds it in automatically.
`git submodule add <url>`	Embed another repository inside yours at a fixed commit.
`git grep "text"`	Search all tracked files, fast.
`.git/hooks/pre-commit`	A script Git runs before each commit (for example, to run a linter). Tools like Husky and pre-commit manage hooks for teams.
`git archive --format=zip HEAD -o out.zip`	Export a clean zip without the .git folder.

Chapter summary

- stash shelves uncommitted work; cherry-pick copies one commit; reflog recovers lost commits.
- blame, log -S, and log --grep investigate history; bisect finds the commit that broke things.
- Aliases turn long commands into short ones.

CHECK YOURSELF

1. You must switch branches urgently but your work is not ready to commit. What do you do?
2. After git reset --hard you realise you lost two commits. How do you find them?
3. What does git cherry-pick do?
4. Which command finds the commit that introduced a bug using a binary search?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 13

Automation with GitHub Actions

In this chapter you will
- understand workflows, events, jobs, steps, and runners
- add a workflow to your repository and watch it run
- make a workflow fail on purpose, then fix it
- use secrets safely

You will need: your hello-git repository on GitHub

13.1 The idea: a robot that runs on every change

Continuous Integration (CI) means every time someone pushes code, a machine automatically builds it and runs the tests, so problems are caught in minutes instead of weeks. **GitHub Actions** is GitHub's built-in system for this. You describe what to run in a YAML file inside `.github/workflows/`, and GitHub runs it on its own computers, free for public repositories (with generous limits for private ones).

Word	Meaning
Workflow	One automation file (.yml) in .github/workflows/
Event (on)	What triggers it: push, pull_request, a schedule, or a manual button
Job	A group of steps that runs on one machine; jobs can run in parallel
Step	One command (**run**) or one reusable action (**uses**)
Runner	The machine that executes the job (ubuntu-latest, windows-latest, macos-latest)
Action	A reusable building block, such as actions/checkout, written by GitHub or the community
Secret	An encrypted value (like an API token) stored in settings, never in your code

HANDS-ON LAB 13.1 Your first workflow

TERMINAL (Git Bash or macOS Terminal)

```
$ cd ~/git-book/hello-git
$ git switch main && git pull
$ mkdir -p .github/workflows
```

Create the file `.github/workflows/ci.yml` in VS Code (`code .`) with this content. **Indentation matters in YAML**: use spaces, never tabs.

.github/workflows/ci.yml

```yaml
name: CI

on:                          # when should this run?
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:         # adds a manual 'Run workflow' button

jobs:
  check:
    runs-on: ubuntu-latest   # the machine GitHub gives you
    steps:
      - uses: actions/checkout@v4          # download your code
```

```
      - name: Say hello
        run: echo "Hello from GitHub Actions"
      - name: List the files
        run: ls -la
      - name: Check required file exists
        run: test -f hello.txt
```

TERMINAL (Git Bash or macOS Terminal)

```
$ git add .github
$ git commit -m "ci: add first workflow"
[main 6a7b8c9] ci: add first workflow
 1 file changed, 17 insertions(+)
 create mode 100644 .github/workflows/ci.yml
$ git push
```

On GitHub open the **Actions** tab. You will see the run appear with a yellow spinner, then a **green check**. Click into it to read each step's log. You just built a robot.

HANDS-ON LAB 13.2 Make it fail, then fix it

Change the last step to look for a file that does not exist, and push:

```
      - name: Check required file exists
        run: test -f missing-file.txt
```

TERMINAL (Git Bash or macOS Terminal)

```
$ git commit -am "ci: break the check on purpose"
$ git push
```

The run now shows a **red cross**, and the failed step is highlighted with its error. Because any non-zero exit code fails a step, **your test commands decide pass or fail**. Change it back to **`hello.txt`**, commit, and push for a green check again.

13.2 A realistic example: test a Node.js project

A TYPICAL CI WORKFLOW FOR NODE.JS

```
name: Node CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [18, 20]          # run the job twice: once per version
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm            # speeds up installs
      - run: npm ci
      - run: npm test
```

> **NOTE** Pin action versions (@v4) so a surprise update cannot break your pipeline. For Python use actions/setup-python, for Java actions/setup-java, and so on.

Secrets

Never write tokens or passwords in a workflow. Store them under **Settings, Secrets and variables, Actions, New repository secret**, then read them in a step:

```
      - name: Deploy
        run: ./deploy.sh
        env:
          API_TOKEN: ${{ secrets.API_TOKEN }}
```

Making CI part of your team rules

Combine Actions with branch protection (Chapter 10): require the **CI** check to pass before a PR can merge. A red cross then physically blocks broken code from reaching main. Add a badge to your README with **`![CI](https://github.com/USER/REPO/actions/workflows/ci.yml/badge.svg)`**.

Common trigger	YAML
On every push to main	on: { push: { branches: [main] } }
On every pull request	on: pull_request
On a schedule (daily at 06:00 UTC)	on: { schedule: [{ cron: '0 6 * * *' }] }
Manual button	on: workflow_dispatch
When a release is published	on: { release: { types: [published] } }

Chapter summary

- A workflow is a YAML file in .github/workflows/; events trigger jobs made of steps on runners.
- Non-zero exit code means a failed step, so your tests decide pass or fail.
- Use secrets for tokens; require passing checks with branch protection.

CHECK YOURSELF

1. Where do workflow files live?
2. What makes a step fail?
3. Where should an API token used by a workflow be stored?
4. What does a matrix do?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 14

GitHub beyond code

In this chapter you will
- publish a website for free with GitHub Pages
- build a profile README and a strong repository front page
- use projects, releases, licences, and the GitHub CLI
- know what Codespaces, Dependabot, and Gists are for

You will need: your GitHub account and a terminal

14.1 GitHub Pages: free hosting

GitHub Pages turns a repository of HTML, CSS, and JavaScript into a live website at **`https://YOUR-USERNAME.github.io/REPO/`**, free. It is the fastest way to put a portfolio online.

HANDS-ON LAB 14.1 Publish a website

```
TERMINAL (Git Bash or macOS Terminal)
```

```
$ cd ~/git-book
$ mkdir my-site && cd my-site
$ git init
$ cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
  <head><title>My First Site</title></head>
  <body>
    <h1>Hello from GitHub Pages!</h1>
    <p>Built with Git and GitHub.</p>
  </body>
</html>
EOF
$ git add . && git commit -m "feat: first web page"
[main (root-commit) 9a8b7c6] feat: first web page
$ gh repo create my-site --public --source=. --remote=origin --push
```

NOTE No GitHub CLI yet? Create an empty repository named **my-site** on github.com, then use **`git remote add origin ...`** and **`git push -u origin main`** exactly as in Chapter 9.

1. On GitHub, open the repository, then **Settings, Pages**.
2. Under **Build and deployment**, choose **Deploy from a branch**, select **main** and **/ (root)**, and Save.
3. Wait a minute or two, refresh, and open the URL shown: **`https://YOUR-USERNAME.github.io/my-site/`**.
4. Edit index.html, commit, push, and watch your site update automatically.

14.2 Your profile README

Create a public repository whose name is **exactly your username** and add a README.md. GitHub shows that file at the top of your profile. Introduce yourself, list your skills, pin your best projects, and link to your portfolio. Employers look here.

14.3 Licences

Code without a licence is legally 'all rights reserved': nobody may reuse it. Add a **LICENSE** file when you create a repository (GitHub offers a picker). Common choices:

Licence	In short
MIT	Very permissive: anyone can use, change, and sell it if they keep your copyright notice. Popular for learning and open source.
Apache 2.0	Permissive like MIT, plus an explicit patent grant.
GPL v3	Copyleft: derivative works must also be released under GPL.
No licence	All rights reserved. Others cannot legally reuse your code.

The site **choosealicense.com** explains the options in plain language. Check the licence of any code you copy.

14.4 Planning tools

- **Issues and labels:** bug, enhancement, documentation, *good first issue* (a signal to beginners).
- **Projects:** Kanban boards and tables with columns like Todo, In progress, Done, linked to issues and PRs.
- **Milestones:** group issues toward a release or deadline.
- **Discussions:** open-ended Q&A and ideas, separate from bug tracking.
- **Wiki:** longer documentation pages (many teams just use a docs/ folder instead).

14.5 Other features worth knowing

Feature	What it does
Codespaces	A complete development environment in your browser; open any repository and start coding with nothing installed
Dependabot	Opens automatic PRs to update outdated or vulnerable dependencies
Secret scanning	Warns you if you accidentally push an API key or token
Gists	Share a single snippet or file quickly (gist.github.com)
Releases	Packaged versions attached to tags, with release notes (Chapter 11)
Stars, watch, follow	Bookmark projects, get notifications, and follow developers you admire
Copilot	AI coding assistance in your editor and on GitHub (see Chapter 16)

14.6 The GitHub CLI (gh)

gh lets you do GitHub tasks without leaving the terminal. Install it with **`winget install --id GitHub.cli`** (Windows) or **`brew install gh`** (macOS), then sign in once with **`gh auth login`**.

Command	What it does
`gh repo create name --public --source=. --push`	Create a GitHub repository from the current folder and push it.
`gh repo clone owner/repo`	Clone a repository by its short name.
`gh pr create --fill`	Open a pull request from your current branch, using your commit messages.
`gh pr list`	List open pull requests.
`gh pr checkout 12`	Download pull request #12's branch so you can test it.

Command	What it does
`gh pr merge 12 --squash`	Merge a pull request.
`gh issue create`	Create an issue interactively.
`gh run list`	Show recent Actions runs; **gh run watch** follows one live.

Chapter summary

• GitHub Pages hosts a static site free from a branch. A repository named like your username gives you a profile README.
• Choose a licence for code you share. Use Issues, Projects, and Milestones to plan.
• gh brings GitHub into the terminal.

CHECK YOURSELF

1. What URL pattern does GitHub Pages use for a project site?
2. What is special about a repository named exactly like your username?
3. Why add a licence to a public repository?
4. What does Dependabot do?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 15

Security and professional hygiene

In this chapter you will
- keep secrets out of repositories, and know what to do when one leaks
- use environment files safely
- protect your account and your commits
- avoid the most common beginner security mistakes

You will need: your hello-git repository

15.1 Git never forgets

Git stores **every version of every file forever**. Deleting a password in a later commit does **not** remove it from history. Anyone who clones the repository can read the old commit. Bots scan GitHub constantly and find leaked keys within minutes.

KEY IDEA If a secret is ever committed or pushed: **(1) rotate it immediately** (create a new one and revoke the old one), and only then (2) clean the history. Rotating is what actually protects you.

15.2 What counts as a secret

- API keys and tokens, passwords, database connection strings, private keys (.pem, id_ed25519), .env files, cloud credentials, and anything customer-related.
- When in doubt: if exposing it would let someone act as you, it is a secret.

15.3 The .env pattern

HANDS-ON LAB 15.1 Keep configuration out of Git the professional way

```
TERMINAL (Git Bash or macOS Terminal)

$ cd ~/git-book/hello-git
$ echo "API_KEY=super-secret-value" > .env
$ echo "API_KEY=put-your-key-here" > .env.example
$ printf ".env\n" >> .gitignore
$ git status -s
 M .gitignore
?? .env.example
$ git add .gitignore .env.example
$ git commit -m "chore: add env example, ignore real .env"
```

Teammates copy **.env.example** to **.env** and fill in their own values. The real **.env** never leaves your computer. Your code reads the value from the environment (for example **`process.env.API_KEY`** in Node or **`os.environ['API_KEY']`** in Python).

15.4 If you leaked a secret

1. **Rotate it now.** Log in to the service, revoke the leaked key, and issue a new one.
2. Remove the secret from the code and use the .env pattern. Commit that fix.
3. If the repository is public or shared, clean the history using **`git filter-repo`** or BFG Repo-Cleaner, then force push with your team's agreement. Everyone must re-clone.
4. Check logs for misuse. Enable GitHub **secret scanning** and push protection so it cannot happen again.

15.5 Protect your account

- Turn on **two-factor authentication** and keep recovery codes safe.
- Prefer **SSH keys** with a passphrase; never share the private key; remove keys of old computers in Settings.
- Give **Personal Access Tokens** the minimum scopes and an expiry date.
- Review third-party apps and Actions before you trust them; pin versions.
- Consider **signing your commits** (SSH or GPG) so GitHub shows them as **Verified**.
- A *private* repository is not a safe place for secrets either: people leave teams and repositories get made public.

Chapter summary

• Git remembers everything: never commit secrets. Use .env (ignored) and .env.example (committed).
• A leaked secret must be rotated immediately; cleaning history alone is not enough.
• Use 2FA, SSH keys with passphrases, minimal-scope tokens, and secret scanning.

CHECK YOURSELF

1. You committed an API key, then deleted it in the next commit. Is it safe?
2. What is the point of an .env.example file?
3. Which is the very first step after leaking a secret?
4. Why give tokens minimal scopes and an expiry date?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 16

Using AI with Git and GitHub, responsibly

In this chapter you will
- use AI tools to work faster on Git tasks
- know what you must always check yourself
- avoid dangerous commands and data leaks

AI assistants such as GitHub Copilot, ChatGPT, and Claude can help you write commit messages, explain errors, and draft workflows. Used well they are accelerators. Used blindly they are a risk. The rule is simple: **you are responsible for every commit, whoever or whatever wrote it.**

16.1 Good uses

Task	How to use AI	What you must still do
Commit messages	Paste the output of **git diff --staged**; ask for a Conventional Commit message	Read it; fix anything inaccurate; keep it short
PR descriptions	Give the diff or commit list; ask for a summary, risks, and a test plan	Verify every claim; add the context only you know
Explaining errors	Paste the exact error text and what you ran	Understand the explanation before acting
Merge conflicts	Paste the conflicted section; ask what each side does	You decide the final behaviour and run the tests
Workflows and .gitignore	Ask for a draft ci.yml or .gitignore for your stack	Review each step; never expose secrets
Learning	Ask 'explain git rebase like I am new' and request examples	Practise it yourself in a throwaway repo

16.2 Rules of the road

1. **Never paste secrets**, private keys, tokens, customer data, or code you are not allowed to share.
2. **Test commands in a throwaway repo or branch first.** Be especially wary of reset --hard, clean -fd, push --force, filter-branch, and rm -rf.
3. **If you cannot explain what a command does, do not run it** on work that matters.
4. **Verify.** AI can be confidently wrong about flags, versions, and behaviour. Check against **`git help <command>`** or docs.github.com.
5. **Keep learning the basics.** AI cannot rescue you if you do not understand your own history.

HANDS-ON LAB 16.1 A safe AI-assisted commit

```
TERMINAL (Git Bash or macOS Terminal)

$ echo "Contact: you@example.com" >> readme.txt
$ git add readme.txt
$ git diff --staged            # copy this output into your AI tool, with no secrets in it
diff --git a/readme.txt b/readme.txt
...
+Contact: you@example.com
$ git commit -m "docs: add contact line to readme"    # message you reviewed and edited
```

Chapter summary

- AI is great for drafts, explanations, and learning; you remain responsible for every commit.
- Never paste secrets; test risky commands on a throwaway branch; verify against real docs.

CHECK YOURSELF

1. What must you never paste into an AI tool?
2. An AI suggests git reset --hard and git clean -fd to 'fix' your repo. What do you do?
3. Who is responsible for a commit written by an AI?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 17

Troubleshooting guide

In this chapter you will
- decode the most common Git and GitHub error messages
- recover from the usual mistakes calmly

Stay calm. Read the **whole** message: Git usually tells you the fix in its **hint:** lines. Run `git status` before anything else.

17.1 Error message decoder

Message	What it means	Fix
fatal: not a git repository	You are not inside a repo folder	cd into the project, or run git init
Author identity unknown / Please tell me who you are	user.name and email are not set	git config --global user.name and user.email
nothing to commit, working tree clean	No changes to commit	Edit a file, or check you saved it
error: pathspec 'x' did not match any file(s)	Wrong branch or file name	git branch -a; check spelling
Your local changes would be overwritten by checkout/merge	Uncommitted edits clash with the switch	Commit them or git stash, then retry
CONFLICT (content): Merge conflict in file	Both sides edited the same lines	Edit file, remove markers, git add, git commit (Chapter 7)
! [rejected] main -> main (fetch first / non-fast-forward)	Remote has commits you lack	git pull --rebase, then git push (never force)
fatal: refusing to merge unrelated histories	The two repos began separately (e.g. README created on GitHub)	git pull origin main --allow-unrelated-histories
fatal: remote origin already exists	You ran remote add twice	git remote set-url origin <url>
error: src refspec main does not match any	No commits yet, or branch is named master	Make a commit; git branch -M main
Permission denied (publickey)	GitHub does not recognise your SSH key	Chapter 9: ssh-add the key; upload the .pub to GitHub; ssh -T git@github.com
Support for password authentication was removed	Passwords no longer work	Use SSH, a token, or gh auth login
You are in 'detached HEAD' state	HEAD points at a commit, not a branch	git switch -c new-branch to keep work; git switch main to leave
warning: LF will be replaced by CRLF (Windows)	Line-ending conversion notice	Harmless; git config --global core.autocrlf true
Vim opens and you cannot exit	Default editor is Vim	Esc, then :wq Enter; set core.editor to code --wait

17.2 Panic button: how to undo common mistakes

I...	Do this
committed to the wrong branch	git reset --soft HEAD~1, git stash, git switch right-branch, git stash pop, commit again (if not pushed)
made a typo in my last commit message	git commit --amend -m "new message" (before pushing)
forgot a file in my last commit	git add file; git commit --amend --no-edit (before pushing)
deleted a branch by mistake	git reflog, find its last commit, git branch name hash
lost commits after reset --hard	git reflog, then git branch rescue hash
merged badly	git merge --abort (mid-merge) or git reset --hard ORIG_HEAD (just after)
pushed a bad commit	git revert hash, then push (safe for shared history)
committed a secret	Rotate it first; then see Chapter 15
committed a file that should be ignored	git rm --cached file; add to .gitignore; commit
want to know where everything is	git status; git branch -vv; git log --oneline --graph --all

PRO TIP When truly stuck, **copy your whole folder** to a backup (**`cp -r project project-backup`**) before trying anything drastic. Then experiment freely.

CHECK YOURSELF

1. A push is rejected with 'fetch first'. Fix?
2. You see 'Permission denied (publickey)'. What is probably wrong?
3. What is the first command to run when something looks wrong?

Answers are in Appendix E at the end of the book. Try before you peek.

CHAPTER 18

Habits of professionals

In this chapter you will
- adopt the daily habits that prevent most problems
- plan what to learn next

18.1 Daily habits

Area	Habit
Commits	Small, focused, and complete; one idea each. Meaningful Conventional Commit messages.
Branches	One branch per task; descriptive names; short-lived; delete after merging.
Syncing	Pull before you start work and before you push. Run git status constantly.
Pull requests	Small, described, linked to an issue, self-reviewed before asking others.
Reviews	Kind, specific, timely. Review code, not people.
Safety	Never commit secrets; never force-push shared branches; test risky commands on a copy.
Repository	Every project has a README, .gitignore, and LICENSE; tests run in CI.
Learning	Break things in a throwaway repo on purpose, then recover with reflog.

18.2 Your learning path

1. Finish every lab in this book and the three capstone projects in Appendix D.
2. Make a pull request to a real open-source project within two weeks (look for *good first issue* labels).
3. Pin three repositories to your profile and write a good profile README.
4. Learn interactive rebase and Git hooks; add CI to every project you build.
5. Teach someone else. Explaining Git is the best way to master it.

KEY IDEA Git is not about memorising commands. It is about **never being afraid of changing your code again**. Commit early, commit often, and when in doubt run **`git status`** and **`git reflog`**.

APPENDIX A

Command quick reference

One-page-per-topic summary of every command in the book. Placeholders are in <angle brackets>.

Setup and help

Command	What it does
`git --version`	Show the installed version.
`git config --global user.name "Name"`	Set your name for commits.
`git config --global user.email "you@x.com"`	Set your email for commits.
`git config --global --list`	Show your global settings.
`git help <command>`	Open the manual for a command.

Create and inspect

Command	What it does
`git init`	Make the current folder a repository.
`git clone <url>`	Copy a remote repository.
`git status / git status -s`	Show state (long / short).
`git log --oneline --graph --all`	Show history as a graph.
`git diff / git diff --staged`	Show unstaged / staged changes.
`git show <commit>`	Show one commit.
`git blame <file>`	Who changed each line.

Stage and commit

Command	What it does
`git add <file> / git add . / git add -p`	Stage a file / everything / chunk by chunk.
`git commit -m "msg"`	Commit staged changes.
`git commit -am "msg"`	Stage tracked changes and commit.
`git commit --amend`	Rewrite the last commit (before pushing).
`git rm <file> / git rm --cached <file>`	Delete and stage / stop tracking, keep on disk.
`git mv <old> <new>`	Rename or move a file.

Undo

Command	What it does
`git restore <file>`	Discard uncommitted edits (permanent).
`git restore --staged <file>`	Unstage a file.
`git reset --soft HEAD~1`	Undo last commit, keep changes staged.
`git reset HEAD~1`	Undo last commit, keep changes unstaged.

Command	What it does
`git reset --hard HEAD~1`	Undo last commit and discard changes.
`git revert <commit>`	New commit that reverses an old one (safe for shared history).
`git clean -n / git clean -fd`	Preview / delete untracked files.

Branches, merge, rebase

Command	What it does
`git branch / git branch -a / git branch -vv`	List branches (local / all / detailed).
`git switch <b> / git switch -c <b>`	Move to / create and move to a branch.
`git branch -d <b> / -D <b>`	Delete merged / force-delete a branch.
`git merge <b>`	Merge branch b into the current branch.
`git merge --abort`	Cancel a conflicting merge.
`git rebase <b>`	Replay your commits on top of b.
`git rebase -i HEAD~3`	Interactively edit the last 3 commits.
`git rebase --continue / --abort`	Continue / cancel a rebase.

Remotes

Command	What it does
`git remote -v`	List remotes.
`git remote add origin <url>`	Connect a remote.
`git push -u origin <branch>`	Push and set tracking.
`git fetch / git fetch --prune`	Download without merging / and clean deleted branches.
`git pull / git pull --rebase`	Fetch and merge / fetch and rebase.
`git push --force-with-lease`	Safer force push, your own branch only.

Advanced

Command	What it does
`git stash push -m "msg" / pop / list`	Shelve work / restore it / list shelves.
`git cherry-pick <hash>`	Copy one commit here.
`git reflog`	Log of everywhere HEAD has been; recovery tool.
`git tag -a v1.0.0 -m "msg" / git push origin v1.0.0`	Create / push a release tag.
`git bisect start / good / bad / reset`	Binary-search for a bad commit.
`git log -S"text" / --grep="text"`	Search history by content / message.

APPENDIX B

Terminal cheat sheet: Git Bash vs macOS

Almost everything is identical because both use a Unix-style shell. These are the differences that matter.

Task	Windows (Git Bash)	macOS (Terminal)
Open the terminal	Start menu, Git Bash; or right-click a folder, Open Git Bash here	Cmd+Space, type Terminal
Home folder path	/c/Users/yourname	/Users/yourname
Open current folder in file manager	explorer .	open .
Open current folder in VS Code	code .	code . (install via VS Code: Shell Command)
Copy to clipboard	clip < file	pbcopy < file
Copy / paste text in terminal	Ctrl+Insert / Shift+Insert, or right-click	Cmd+C / Cmd+V
Install tools	winget install ...	brew install ...
Line endings setting	core.autocrlf true	core.autocrlf input
Credential storage	Git Credential Manager (built in)	credential.helper osxkeychain
Add key to SSH agent	ssh-add ~/.ssh/id_ed25519	ssh-add --apple-use-keychain ~/.ssh/id_ed25519
Stop a running command	Ctrl+C	Ctrl+C
Clear the screen	Ctrl+L or clear	Cmd+K or clear

Do not use Windows Command Prompt or PowerShell for this book: commands such as **`touch`**, **`cat`**, **`ls`**, and **`rm`** behave differently there.

APPENDIX C

Glossary

Term	Meaning
Repository (repo)	A project folder tracked by Git, with its full history
Commit	A saved snapshot with author, date, message, and parent
Hash (SHA)	The unique fingerprint of a commit or object, such as 3f2a9c1
Branch	A movable label pointing at a commit
HEAD	Pointer to your current position, usually the current branch
Working tree / directory	The actual files on disk that you edit
Staging area (index)	Where changes wait to be included in the next commit
Merge	Combine another branch's history into the current one
Fast-forward	A merge that just moves the pointer forward, with no merge commit
Rebase	Replay commits on top of another base, rewriting them
Conflict	Overlapping edits that Git cannot combine automatically
Remote / origin	A hosted copy of the repository / its default name
Upstream	The original repository that your fork came from
Tracking branch	A local branch linked to a remote branch
Clone	A full local copy of a remote repository
Fork	Your own copy of someone else's repository on GitHub
Fetch / Pull / Push	Download only / download and integrate / upload
Pull request (PR)	A proposal to merge a branch, with review and checks
Issue	A tracked task, bug, or idea on GitHub
Stash	A temporary shelf for unfinished changes
Cherry-pick	Copy one commit onto another branch
Reflog	A private log of everywhere HEAD has been; recovery tool
Tag / Release	A permanent label on a commit / a published version on GitHub
Detached HEAD	HEAD pointing straight at a commit instead of a branch
CI/CD	Automated testing and delivery on every change
Workflow / Runner	A GitHub Actions automation file / the machine that executes it
Protected branch	A branch with rules such as required reviews and passing checks
Squash	Combine several commits into one
Markdown	A simple text format used for READMEs and comments
SSH key	A key pair that proves your identity to GitHub without a password

APPENDIX D

Capstone projects

Three projects that pull the whole book together. Put each on your GitHub profile.

Project 1: Personal portfolio site

1. Create a repository **portfolio** with an index.html, a style.css, and a README, using small Conventional Commits.
2. Work on a branch **feature/projects-section**, push, open a PR to your own main, review it yourself on the Files changed tab, and squash-merge.
3. Add a LICENSE and a .gitignore. Enable GitHub Pages and put the live link in the README.
4. Add a CI workflow that checks that index.html exists. Tag **v1.0.0** and publish a release.

Project 2: Team simulation with a partner

1. Create a shared repository and add your partner as a collaborator. Protect main: require a PR and one approval.
2. Each of you creates three issues and works on separate branches. Edit one shared file on purpose to cause a conflict.
3. Review each other's PRs with line comments and suggestions. Resolve the conflict inside a PR.
4. Link PRs to issues with 'Closes #N'. Finish with a release v0.1.0.

Project 3: Open-source contribution

1. Find a beginner-friendly project with the label **good first issue** (see Appendix F).
2. Fork, clone, add upstream, read CONTRIBUTING.md, and comment on the issue to claim it.
3. Create a branch, make a small fix (a typo or documentation counts), and follow the project's commit style.
4. Open a clear pull request, respond to feedback politely, and celebrate your merge.

APPENDIX E

Answer key

Check your answers to the questions at the end of each chapter.

Chapter 1

#	Question	Answer
1	In one sentence, what is the difference between Git and GitHub?	Git is the version control tool that runs on your computer; GitHub is a website that hosts Git repositories and adds collaboration features such as pull requests and Actions.
2	What does it mean that Git is 'distributed'?	Every developer has a complete copy of the whole history, so most work can be done offline and no single server is a point of failure.
3	Why was Git created?	In 2005 the Linux kernel community lost its free licence for BitKeeper and Linus Torvalds wrote a fast, distributed tool to replace it.
4	Name three benefits of using version control.	For example: go back to old versions, see who changed what and why, work in parallel safely, and experiment without fear.

Chapter 2

#	Question	Answer
1	Which command shows the folder you are currently in?	pwd
2	What is the difference between > and >> when writing to a file?	> replaces the file's contents; >> appends to the end.
3	Why does Git need your name and email?	Every commit records its author. Using the same email as your GitHub account links commits to your profile.
4	You are stuck in Vim. How do you quit and save?	Press Esc, then type :wq and press Enter.

Chapter 3

#	Question	Answer
1	What are the four places a change can live, in order?	Working directory, staging area, local repository, remote repository.
2	Why are branches cheap in Git?	A branch is only a small pointer to a commit; nothing is copied.
3	What does HEAD mean?	A pointer to where you currently are, usually the current branch (and so its latest commit).
4	What happens to the hashes of later commits if you change an old commit?	They all change, because each hash is calculated from the content and the parent hash.

Chapter 4

#	Question	Answer
1	What does 'working tree clean' mean?	Your files match the last commit; there is nothing to stage or commit.
2	What is the difference between git diff and git diff --staged?	git diff shows unstaged changes; git diff --staged shows changes staged for the next commit.

#	Question	Answer
3	Write a good commit message for fixing a crash when a username is empty.	For example: fix(auth): handle empty username on login.
4	You committed .env by mistake and added it to .gitignore. Is it now ignored?	No. .gitignore only affects untracked files. Run git rm --cached .env and commit; and if it held secrets, rotate them.

Chapter 5

#	Question	Answer
1	You staged a file by mistake. Which command unstages it and keeps your edits?	git restore --staged
2	What is the difference between git reset --soft HEAD~1 and git reset --hard HEAD~1?	Soft undoes the commit but keeps the changes staged; hard undoes it and discards the changes from your files too.
3	A bad commit is already pushed to GitHub. Reset or revert?	Revert, because it adds a new commit and does not rewrite shared history.
4	What does git clean -n do?	A dry run that lists untracked files git clean would delete, without deleting anything.

Chapter 6

#	Question	Answer
1	Which command creates a branch and switches to it in one step?	git switch -c (or the older git checkout -b).
2	Why did greeting.txt disappear when you switched to main?	It only exists on the feature branch; switching branches changes the files in your folder to match that branch's latest commit.
3	What command shows the branch structure as a graph?	git log --oneline --graph --all
4	You are in a detached HEAD and made a good commit. How do you keep it?	Create a branch there with git switch -c .

Chapter 7

#	Question	Answer
1	You want feature to be merged into main. On which branch do you run git merge feature?	On main (the receiving branch).
2	When does Git do a fast-forward merge?	When the receiving branch has no new commits since the other branch was created, so the pointer can simply move forward.
3	What do the lines <<<<<<<, =======, and >>>>>>> mean in a conflicted file?	Start of the current branch's version, the divider, and the end of the incoming branch's version. They must all be deleted after you choose the final text.
4	You are halfway through a conflicting merge and want out. Which command?	git merge --abort

Chapter 8

#	Question	Answer
1	Why do rebased commits get new hashes?	Their parent commit changed, and a commit's hash depends on its content and parent.

#	Question	Answer
2	State the golden rule of rebasing.	Never rebase commits that other people already have (shared or public history).
3	In an interactive rebase, which word turns a commit into part of the previous one and discards its message?	fixup
4	You hit a conflict during a rebase and fixed the file. What two commands come next?	git add , then git rebase --continue.

Chapter 9

#	Question	Answer
1	What is the difference between git fetch and git pull?	fetch downloads new commits without changing your files; pull fetches and then merges (or rebases) them into your current branch.
2	What does 'origin' mean?	The default name for the remote repository you cloned from or first connected.
3	Your push is rejected with 'fetch first'. What should you do?	Run git pull (or git pull --rebase) to integrate the remote's commits, resolve any conflicts, then push again. Do not force push.
4	Which key file must you never share, the .pub or the one without it?	The one without .pub (the private key).

Chapter 10

#	Question	Answer
1	What does writing 'Closes #12' in a PR description do?	It automatically closes issue 12 when the PR is merged.
2	Why do open-source contributors fork instead of pushing to the original repository?	They do not have write permission; a fork gives them their own copy where they can push and then open a PR.
3	What is the difference between origin and upstream?	origin is your own fork (or the repo you cloned); upstream is the original repository you forked from.
4	Name the three ways to merge a pull request on GitHub.	Create a merge commit, squash and merge, rebase and merge.

Chapter 11

#	Question	Answer
1	Which workflow uses main, develop, feature, release, and hotfix branches?	Git Flow.
2	Write a Conventional Commit for adding a dark mode feature.	For example: feat(ui): add dark mode.
3	Your project is at 2.3.4. You fix a small bug without changing behaviour. What is the next version?	2.3.5 (a patch release).
4	Are tags pushed to GitHub automatically with git push?	No. Push them with git push origin or git push origin --tags.

Chapter 12

#	Question	Answer
1	You must switch branches urgently but your work is not ready to commit. What do you do?	git stash push -m "msg", switch, do the other work, switch back, then git stash pop.
2	After git reset --hard you realise you lost two commits. How do you find them?	git reflog shows the earlier hashes; then git branch rescue or git reset --hard .
3	What does git cherry-pick do?	Copies that one commit onto your current branch as a new commit.
4	Which command finds the commit that introduced a bug using a binary search?	git bisect (start, mark good and bad, or git bisect run).

Chapter 13

#	Question	Answer
1	Where do workflow files live?	In .github/workflows/ in your repository, as .yml files.
2	What makes a step fail?	Its command exiting with a non-zero status code.
3	Where should an API token used by a workflow be stored?	In GitHub Secrets (Settings, Secrets and variables, Actions), referenced as ${{ secrets.NAME }}; never in the code.
4	What does a matrix do?	Runs the same job several times with different values, such as Node versions or operating systems.

Chapter 14

#	Question	Answer
1	What URL pattern does GitHub Pages use for a project site?	https://USERNAME.github.io/REPOSITORY/
2	What is special about a repository named exactly like your username?	Its README.md is displayed on your GitHub profile page.
3	Why add a licence to a public repository?	Without one, the code is legally 'all rights reserved' and nobody may reuse it.
4	What does Dependabot do?	Automatically opens pull requests to update outdated or vulnerable dependencies.

Chapter 15

#	Question	Answer
1	You committed an API key, then deleted it in the next commit. Is it safe?	No. It is still in history. Rotate (replace) the key immediately, then clean history if needed.
2	What is the point of an .env.example file?	It documents which variables are needed, with fake values, and is committed; the real .env is ignored and stays local.
3	Which is the very first step after leaking a secret?	Rotate it: revoke the leaked secret and create a new one.
4	Why give tokens minimal scopes and an expiry date?	To limit the damage if a token leaks.

Chapter 16

#	Question	Answer
1	What must you never paste into an AI tool?	Secrets, private keys, tokens, customer data, or code you are not allowed to share.
2	An AI suggests git reset --hard and git clean -fd to 'fix' your repo. What do you do?	Do not run them blindly; understand them, test on a copy or throwaway branch, and check you have no uncommitted work you need.
3	Who is responsible for a commit written by an AI?	You are.

Chapter 17

#	Question	Answer
1	A push is rejected with 'fetch first'. Fix?	git pull --rebase (or git pull), resolve conflicts, then git push.
2	You see 'Permission denied (publickey)'. What is probably wrong?	Your SSH key is not loaded or not added to your GitHub account.
3	What is the first command to run when something looks wrong?	git status.

APPENDIX F

Resources for next steps

Resource	Why it helps
Pro Git (free book): git-scm.com/book	The definitive free reference, by Scott Chacon and Ben Straub
Learn Git Branching: learngitbranching.js.org	A visual, interactive game that makes branching, merging, and rebasing click
GitHub Skills: skills.github.com	Free hands-on courses that run inside GitHub itself
GitHub Docs: docs.github.com	Authoritative guides for every GitHub feature
Conventional Commits: conventionalcommits.org	The commit message convention in full
Choose a License: choosealicense.com	Plain-language help picking a licence
First Contributions: github.com/firstcontributions/first-contributions	A safe practice repo for your first pull request
Good first issues (for example goodfirstissue.dev)	Find beginner-friendly open-source issues
gitignore templates: github.com/github/gitignore	Ready-made .gitignore files for every stack

Well done for reading this far. Keep a throwaway repository open, practise a little every day, and teach someone what you learned. That is how Git becomes second nature.